Where a Defense Intervenes Decides Whether It Survives: Context Poisoning across Security Attacks and Safety Failures

Anonymous Author(s)
Anonymous Institution

***Abstract*—Large language models (LLM) became extremely powerful in this modern era. The capabilities of an LLM scale as the context gets bigger and better, as these agents answer from their context. An attacker who controls the context controls the answer. Two research communities study this failure, and neither tests the other's work. Security researchers study context an attacker changes on purpose; safety researchers study context that becomes wrong on its own. The model cannot tell the difference.**

We read 929 papers in full and recovered every confirmed (defense, attack) pair either community reports. Only a handful cross between them, and exactly 1 security defense has ever been evaluated against a safety failure mechanism. The two sides do not duplicate each other. They divide the problem instead. Security focuses on how the bad text arrives and defends at ingestion. Safety deals with why the model believes it and defends at reasoning. A defense at the reasoning stage judges text the attacker has already chosen.

We then ran the experiments neither community has run. Our campaign covers 110 pairs, each run from its authors' released code and weights against an undefended model on the same items. Where a defense intervenes decides whether it survives. Techniques that act at ingestion or at execution drive attack success to zero in 24 of 56 agent conditions, while no defense that reasons over text already in the window leaves the model better off than no defense at all. We report results for one model family and one retrieval setting.

***Index Terms*—large language model agents, context poisoning, prompt injection, retrieval-augmented generation, knowledge conflict, defense evaluation**

I. Introduction

An LLM agent reads from many places. It reads retrieved documents and the output of the tools it calls. It reads the descriptions of those tools, the files and skills it loads, and messages from other agents. It also reads its own stored memory [1], [2]. All of this arrives in one context window, and the model treats it as one body of text. The model has no reliable way to tell which part of that text is trustworthy and which part is not [3].

It helps to picture the context window as a body of water fed by a fixed set of inflow streams. Whatever enters through any stream mixes into the same body and affects everything downstream, whether it was dumped in deliberately or arrived as runoff. The question this paper answers first is simply, how many streams are there, and how many consequences may arise because of this pollution?

Two research communities study what goes wrong when untrustworthy content gets in, and they use different words for it. The security community studies content that an attacker placed there on purpose. It names attacks such as indirect prompt injection [3], retrieval poisoning [4], [5] and memory backdoors [6], and it publishes defenses against them [7]–[9]. Success means the attacker fails to achieve a goal.

The AI/ML community studies content that became wrong without any attacker. It names mechanisms such as lost-in-the-middle [10] and context rot [11]. It also names knowledge conflict, where a retrieved passage disagrees with what the model already learned [12]. It publishes fixes for them [13]–[15]. Success means accuracy goes up.

The two communities describe the same underlying failure: a passage the model should not trust enters the context window and changes the output. Only the provenance differs; the model sees tokens either way. The difference is not merely one of vocabulary. If it were, a reader could translate between the two communities and lose nothing. They differ in something load-bearing: what counts as a successful evaluation? The AI/ML community measures a fix for the accidental case on an average-case benchmark, because nothing adapts to it. The security community measures a defense for the adversarial case against the worst case, because something does. Those are not two styles of the same test. A method can pass the first test convincingly and fail the second one completely. Neither community's publishing process catches this, because each reviews against its own convention.

So the field publishes, adopts, and cites a method without anyone asking whether it survives the other community's test. That is a testable claim about the literature, not a complaint about it. We test it by reading both literatures under one taxonomy and recovering every experiment either side has run. The test finds a gap. We then run the experiments that fall in it to answer the following research questions.

Research questions.

- RQ1 **Are the two communities reinventing the wheel?** We read 929 papers from both sides under one scheme, compare where each community's defenses intervene, and recover every (defense, attack) pair the 709 papers that name a technique report. They are not, and what they do instead is what leaves the region untested (Sections III, III-H).
- RQ2 **Does a defense built for one regime's problem survive the other's?** We execute the pairs the liter-

ature skipped, from the authors' released code and weights (Sections IV, VI).

RQ3 **Is failure a property of individual methods or of a design choice they share?** We build a payload that separates the explanations rather than maximizes damage (Section VI-B).

RQ4 **Do the measurements the field leaves out change the result?** We re-read every result against a score almost no paper reports: what the same model scores with no document at all. We also check the results on a second base model and on a second implementation of the one design that held (Section VI-C).

RQ5 **Does that explanation survive an adaptive attacker?** We build two optimizers against the one method that resisted (Section VI-D).

Both literatures have been systematized before: the security side repeatedly [1], [16], [17], and the AI/ML side on its own terms as a problem of knowledge conflict [18]. None of it runs the experiment. A survey of one literature has only that literature's methods in hand, and the test we need takes a defense from one side and an attack from the other. So we read both, found the pairs nobody has run, and ran them from the authors' released code and weights. To our knowledge this is the first evaluation in which published defenses from one of these communities meet published failure mechanisms from the other.

Scope. Three limits bound what follows. Each setting runs on one dataset, and we plant poisoned passages into the context rather than attack the retriever, so the numbers describe what a generator does with bad text. Most runs use one model family at one size, and our own control shows that a different model can reverse an outcome. Almost every attack we run knows nothing of the defense it meets, which bounds attack strength from below: conservative evidence when a defense fails, and no evidence at all when one holds.

Contributions.

- **We show that the two communities divide the problem rather than duplicate it (RQ1).** One side studies how the bad text arrives and defends where it enters; the other studies why the model believes it and defends where it reasons. That division, not duplication, is what leaves a region neither side has tested: of every confirmed (defense, attack) pair either community reports, only a handful cross between them, and a security defense has been evaluated against an AI/ML failure mechanism exactly 1 time (Sections III-H, III-G).
- **We build a platform and run 110 pairs on it (RQ2).** Every method executes against an undefended arm on the same items, under controls we fixed before any run (Section IV).
- **We trace the failures to a single design choice (RQ3).** Every method that fails weights the retrieved text against the model's memory, and turning that weight relocates correctness instead of creating it. What they take from memory the attacker receives, and what they take from the attacker memory receives, so neither direction moves anything toward the real document (Sections VI, VI-B).
- **We re-read every result against a score the field omits, and one finding does not survive it (RQ4).** Without a no-document score, a defense that never uses the document looks robust. A second base model and a second implementation leave the other results unchanged (Section VI-C).
- **We build two adaptive attacks against the one defense that resisted (RQ5).** It loses 58% of its protection to an attacker who reads its rule, which prices the method rather than dismissing it (Section VI-D).
- **We state what a deployment inherits from these methods.** A method that tunes how much the model trusts retrieved text is correct when the text is merely stale and dangerous when an attacker owns it, and a writable corpus is the second case (Section VII).
- **We derive the design rule a defense covering both regimes would have to follow, and we have not yet built one.** Every signal inside the context window belongs to whoever wrote the passage, which is why the reasoning stage fails and why the one method that resisted fell to an attacker who read its rule. Such a method has to key on something the attacker does not hold — provenance, agreement across independently retrieved passages, or the decision to act rather than the decision to believe — and it still needs the reasoning stage for the stale document nobody wrote as an attack. The experiment that tests this is specified and queued; it has not run, and we claim no result from it (Section VII).
- **We release a runnable artifact.** Every attack and defense executes from its authors' released code at a pinned commit, verified unmodified; the harness, the payloads and the full census ship with it.

How the paper reads. Section II fixes the terms and the threat model. Section III answers RQ1 and ends at the region nobody has tested. Section IV builds the harness and Section VI runs it. That section answers RQ2 and, in Section VI-B, RQ3; it then tries to break its own results, with the controls the literature omits (RQ4) and with an adaptive attacker (RQ5). Section VII turns what survives into conventions, into what a deployment inherits, and into the design rule a defense covering both regimes would have to follow. Section VIII states the limits and what is not finished.

II. Background and threat model

This section fixes the terms the rest of the paper uses. It defines the failure we study, shows that two communities name it twice, states the system and the adversary we assume, and explains the vocabulary the later sections depend on.

A. Context poisoning

Context poisoning is our term for three things happening together. Content enters an agent's context window. The agent should not have trusted it. The agent's behavior changes as a result. The definition says nothing about who put the content

there, and that silence is deliberate: it is what lets both communities fit inside one term.

A poisoned agent's output becomes another agent's input [19], and poisoned memory reactivates in a later session [6]. The term is wider than prompt injection, which is one channel and one intent. It is wider than knowledge conflict, which is one consequence. It is narrower than "LLM safety", because the damage must arrive through the context window. Two things fall outside it. Training-time poisoning and weight attacks do not travel through the context. Jailbreaks in which the user is the attacker have no third-party content at all.

B. One failure, two vocabularies

That one failure has two names, because two communities arrived at it separately. Figure 1 puts the two vocabularies side by side. The slots are the same: the same channel, the same position in the context window, the same downstream consequence. Only the names differ, and one assumption behind them. We take that assumption from Lin et al. [20]. Security work assumes an adversary and asks whether a system resists "intentional attacks … in the presence of adversarial actors". Safety work assumes none and asks whether a system avoids "unintended harmful outcomes … despite uncertainties in inputs, goals, training data, or deployment conditions". We call the condition a paper addresses its **regime**, and every paper in our corpus takes one of these two. A regime is not a **venue**, the conference or journal that published the paper. The two are easy to mix up, because security work often appears at a security venue, but Section III-C shows that adversarial work appears at AI venues just as often. We write *security venue* or *AI venue* for where a paper appeared, and plain security or safety for what it addresses. An adversary re-optimizes against whatever method is in place; a stale document does not. That difference is real, because a method tuned to the average case has no reason to hold at the worst case. It is also smaller than it looks, because the model reads tokens either way, and nothing in the text marks which hand wrote it. Section VI tests the second half with payloads that assert the same false answer, in the same slots, in the same amount, and differ only in whether a person wrote the sentence to be believed.

Two vocabularies for one failure force a choice of words, and this paper makes it once. We use *mechanism* for anything that makes the context stop supporting the right answer, and *fix* for anything meant to keep it supporting that answer, whatever either one's authors call it.

Each word splits in two. A mechanism is an *insertion* when content arrives in the window that should not be believed, and a *degradation* when nothing arrives. In the second case the context stops working because it grew too long, because the decisive passage sits in the wrong place, or because compaction evicted the rule the agent needed. Accuracy falls with the right document present and correctly retrieved. *Attack* sits inside insertion, not beside degradation. It names the 175 insertion mechanisms an adversary wrote, against the 19 that a retriever, a tool or the agent's own memory delivered with nobody behind them. Splitting on the adversary instead would have renamed the regime partition and added nothing, because every security-regime mechanism has one and no safety-regime mechanism does. Insertion cuts across the regimes instead: 176 of the 176 security-regime mechanisms are insertion and 0 are degradation, while the safety regime splits 20 to 40.

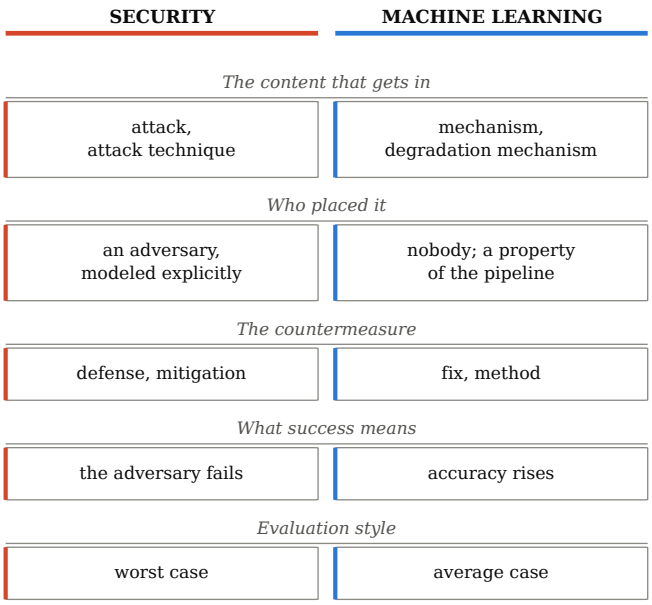


Fig. 1. The same five slots, named twice. The taxonomy in Section III-B uses neutral terms so that both sides fit.

A fix splits by what its own evaluation tested. A *defense* met an adversary there. A *remedy* met only incidental failure. That is 261 defenses and 240 remedies of 541. The split records what a method has been pointed at, not how well it works, and a fix that filters what enters has nothing to act on against degradation.

Two more words name where an answer comes from. The **context** is the document, tool output or message placed in front of the model. **Memory** is what the model learned in training and holds in its weights, which the safety literature calls *parametric memory*. A large family of published fixes works by shifting the model from the second to the first.

C. System and threat model

Both communities study the same system, and only one of them assumes an attacker inside it. The subject of the system is an LLM agent: a model that, within one task, repeatedly reads content it did not author, decides, and acts. Content from many sources shares one context window, with no trustworthy marking of where any of it came from. And an action taken in a corrupted context cannot be undone by fixing the context afterwards.

The adversary. The adversary controls the content of at least one channel. They do not control the model weights, the decoding procedure, or the agent's code. This covers the realistic deployment cases. An adversary can write a document that a retriever may fetch [4]. They can publish a tool whose

description the agent reads [21]. They can post content that a browsing agent visits [3] or contribute a skill file that the agent loads [22]. The adversary knows the defense and may adapt the payload to it. The adversary does not need to know the user's query: every attack we run is query-agnostic by construction.

Where there is no adversary. Without an attacker there is no threat model. The content goes wrong because the pipeline behaves normally. A retrieved document is out of date. A passage is close to the topic but does not answer the question [23]. Important text sits in the middle of a long input [10]. A retrieved fact disagrees with what the model learned in training [18]. These causes do not adapt, so this literature evaluates on average-case benchmarks.

III. Two literatures, one problem

Two communities work one problem and do not cite each other. The obvious question, and RQ1, is whether they are doing each other's work twice. The answer is no, and what they are doing instead costs the field more than duplication would. Everything in this section is measurement over our corpus, and it ends at the cell of the matrix that is empty.

A. Corpus

We began from 51 seed papers found during scoping, queried the Semantic Scholar Academic Graph and arXiv APIs with keyword clusters drawn from both vocabularies, and snowballed forward and backward. The discovery pool reached 25922 papers. 10975 of those are hop-2 records, discovered before the snowball depth was capped at one hop and kept in the database but outside the working pool, which leaves 14947. Dual coding at that scale is not feasible for a small team, so we screened on title and abstract against written inclusion criteria, then read the survivors in full. The included corpus is 929 papers, each carrying 35 recorded columns of which 20 are coded from the full text.

Screening and synthesis are different sets, and we keep them apart. A paper can be about context poisoning and still name no attack and no defense: surveys, benchmarks, practitioner guides, applications of somebody else's method. 220 of the 929 do exactly that, so the census, the taxonomy cube and the coverage matrix are computed over the 709 papers that contributed a named technique, and the citation analysis is computed over all 929. Figure 2 reports both counts at its last stage. AgentDojo [24] is the clearest case. It is the environment 56 of our own pairs run in, and it names neither an attack nor a defense. Coding it to a regime and a cell asserted something about it that is not true.

Of the 220, 40 are surveys or systematizations, 18 are benchmarks and evaluations, 18 are framework or architecture papers and 4 are position papers. The remaining 140 match none of those descriptions by title: they are applications, empirical studies of off-the-shelf methods, and practitioner guides. Section VIII reports a recall pass over that remainder.

We follow PRISMA [25] for identification, screening and inclusion, and Figure 2 reports the count at every stage. The

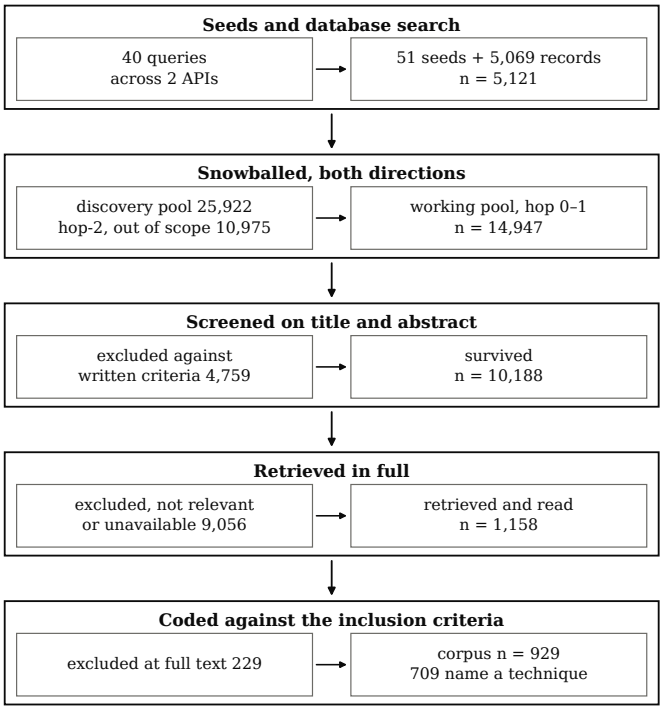


Fig. 2. Corpus construction. Each stage reports what it removed on the left and what it passed to the next stage on the right. The queries, their dates and their hit counts ship with the artifact as a search log.

artifact ships the 40 queries with their dates and their hit counts.

Screening ran as two passes over title and abstract. The first applied the written criteria to the whole working pool and removed 4759 records. The second read the 10188 survivors and carried 1158 of them forward to full-text retrieval. A record left at the second pass on either of two grounds. Its title and abstract placed it outside the scope we had written down, or no full text was obtainable from a peer-reviewed venue or a recognized preprint server, which is the case for withdrawn papers and for records whose only version was an uncredentialed copy. Reading an uncredentialed copy would put a claim in the census that no venue stands behind.

We then coded the 1158 retrieved papers against the inclusion criteria and kept 929. The paper stays if it contributes an attack, a failure mechanism, a defense, or a fix. Benchmarks, surveys, measurement studies, and position papers stay even though they propose no technique, because they are how the field is organized. That pass removed 53 papers and recovered 34 techniques the first extraction had missed, so the registry counts here are floors.

B. A joint taxonomy

We place every named mechanism and every named defense on three axes. Each axis value is named by the work that first identified it as a distinct surface or a distinct harm; we group the names rather than coin them. The axes let the two literatures land in the same space without forcing them into the same words. A security attack and a safety mechanism that

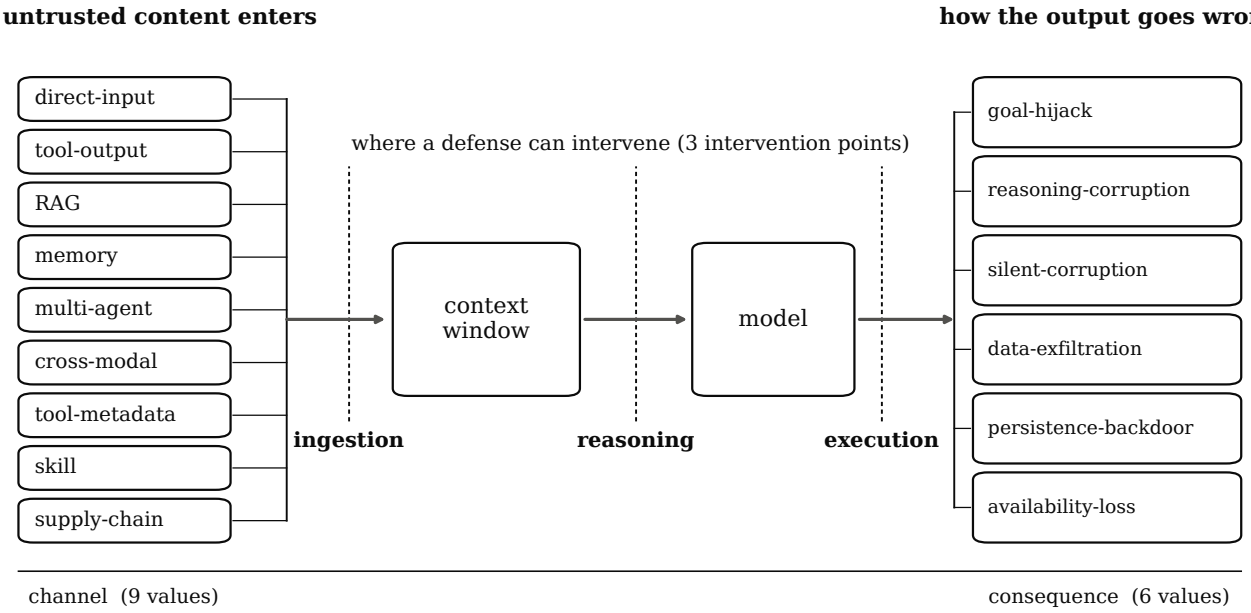


Fig. 3. The taxonomy against the system it describes. Untrusted content arrives through one of 9 **channels**, enters one context window, and the output fails as one of 6 **consequences**. A defense can intervene at one of three points between them.

enter by the same route and cause the same damage occupy the same cell. They differ only on intent. That is what makes our taxonomy a way of finding missing experiments rather than a filing scheme.

Channel is the part of the context window at issue: the route the content took in for an insertion mechanism, and the stream that grew, shifted or was evicted for a degradation one. There are 9. direct-input is content placed in the prompt by the user or an upstream service [26]. RAG is a passage a retriever selected from a corpus [4], [5]. tool-output is whatever a called tool returned [3]. tool-metadata is the names, descriptions and schemas of the tools themselves, which the model reads before it calls anything [21]. memory is what the agent stored from an earlier turn or session [27]. skill is a file or instruction set the agent loads to extend itself [22]. multi-agent is a message from another agent [28]. cross-modal is content that arrives as an image, audio or document and is rendered into text [29]. supply-chain is content entering through a package, template or model the agent depends on [30].

Intent is whether the content was placed deliberately: adversarial, safety, or studied as both.

Consequence is what the agent then does wrong. There are 6. goal-hijack is pursuing an objective it was not given [3]. reasoning-corruption is reaching a wrong conclusion while pursuing the right goal [10]. silent-corruption is producing a wrong output with nothing to signal it [5]. data-exfiltration is information crossing a boundary it should not [31]. persistence-backdoor is corruption that survives into a later session [30]. availability-loss is the agent not delivering its work: either it fails to finish the task at all [11], or it finishes only after spending compute, money or quota out of proportion to it [32], [33]. Figure 3 sets the three axes against the system they describe.

51 of the 108 cells (47.2%) hold no named attack and no named defense from either literature. The cube counts what the field has named, not which papers discuss a cell. A survey that ranges over six channels therefore occupies none of them. Emptiness is not uniform, and Figure 4 puts the named attacks beside the named defenses in each cell. Three things are visible there that a count of papers hides.

The two panels barely overlap. The security literature holds 176 of the 236 attacks and masses them in goal-hijack; the safety literature holds 60 and masses them in reasoning-corruption and silent-corruption. The columns each community works are nearly disjoint, which is the same separation the citation rates show, drawn as a picture.

Effort is concentrated rather than scarce. Four cells hold more than fifty defenses each — tool-output $\times$ goal-hijack at 113, direct-input $\times$ goal-hijack at 61, RAG $\times$ silent-corruption at 60 and memory $\times$ reasoning-corruption at 58 — and between them they absorb 54% of the 541 defenses in the census.

And 8 cells hold a named attack with no defense ever proposed against it. 7 of them are in the security panel, 4 of those are persistence-backdoor, and the last is memory $\times$ data-exfiltration on the safety side. A cell one community has to itself is a gap; a cell with attacks and no defense is a different and sharper one.

The totals column collapses the consequence axis and reports it. multi-agent and memory carry several defenses per mechanism, while supply-chain has fewer defenses than mechanisms and cross-modal, tool-metadata and skill sit

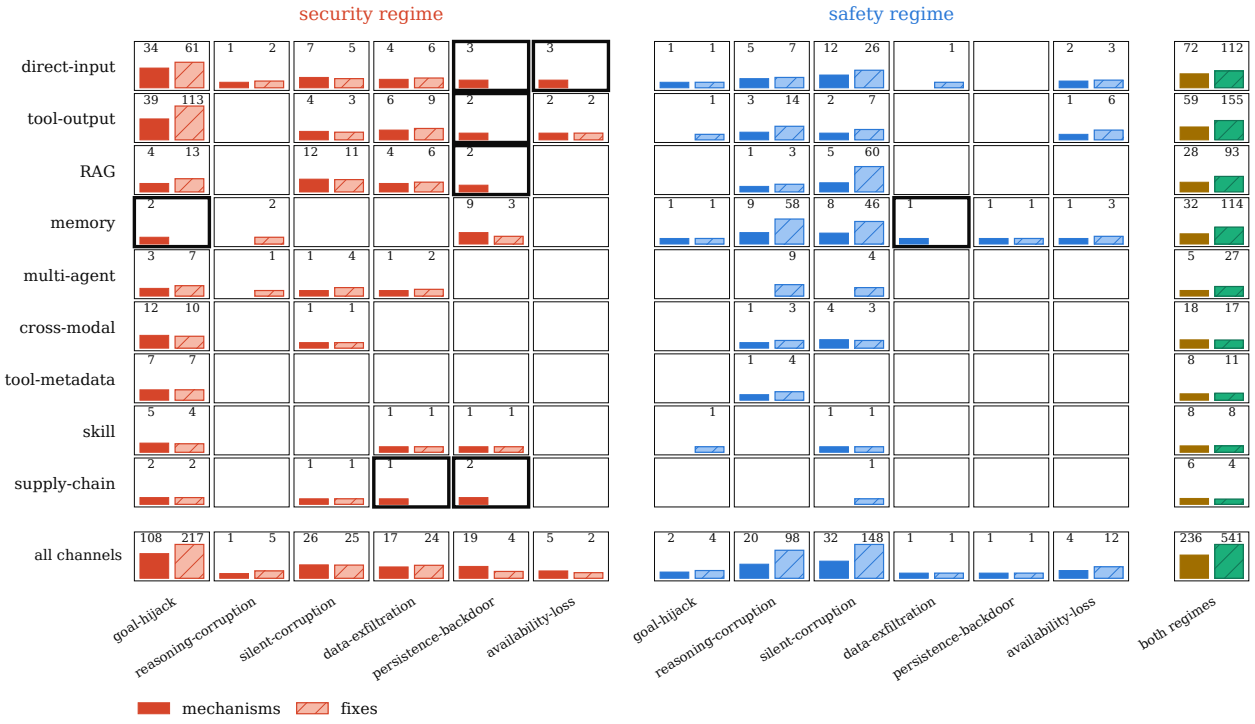


Fig. 4. Mechanisms against fixes in every channel × consequence cell, split by regime. One convention throughout: a solid bar is a mechanism and a hatched bar is a fix, in cells and in totals alike, each with its count printed above it on a fixed line. Hue is the regime, so a panel's own totals keep that panel's hue; only the last column, which adds the regimes together, leaves them. A cell that holds mechanisms and no fix is outlined in bold. Each panel's last row totals a consequence down that panel, the last column totals a channel across both, and totals carry their own scale. All 236 mechanisms and all 541 fixes place, so the two panels together are the whole census.

at parity. Those are the routes by which an agent accepts capability from a third party, and they are the thinnest in the corpus.

Census. The registry holds 236 named mechanisms, 176 from the security literature and 60 from the safety literature, and 541 named defenses and fixes, 277 and 264 respectively. By consequence, goal-hijack dominates the threat side at 110 (46.6%), followed by reasoning-corruption at 21 (8.9%). By channel, direct-input (72) and tool-output (59) together hold close to three fifths. On the remedy side, 206 methods act at reasoning time (38.1%), 154 at ingestion, 79 at execution, and 102 specify no single point.

Almost every defense answers to one validation regime. 261 were validated in the security regime only and 240 in the safety regime only, leaving 14 (2.6%) validated against both and 26 with no validation we could identify. So 97% of published defenses carry an untested cross-regime question, which is the gap this paper goes on to measure and then to fill.

The regime a defense was validated in is coded separately from the literature it was published in, and the two turn out to be nearly the same partition. Of 541 defenses, only 3 were ever validated in the other community's regime. So the empty cell of Section III-G, which is a statement about tested pairs, has a counterpart one level down in the methods themselves.

The two sides are also not the same size. There are 541 remedies for 236 problems, and yet 132 of 250 named mechanisms (52.8%) have no defense ever evaluated against them. Effort is concentrated rather than scarce: a single umbrella entry, indirect prompt injection, absorbs more tested pairs than any other entry by a wide margin, while 34 of 64 named injection variants have none at all. The field tests heavily against a category and lightly against its members.

C. Regime versus venue

The regime says what a paper is about, either security or safety. Venue says where its authors sent it. We code the two independently, which is what separates the division we draw from the field's own institutional one.

The field does not divide itself this way institutionally. Of the 101 security-regime papers that name a venue, 53 published at an AI venue and 48 at a security venue. Adversarial work on LLM agents goes to ACL, EMNLP, NeurIPS and AAAI about as readily as to S&P and CCS.

The traffic runs one way. Of the 80 safety-regime papers that name a venue, 0 published at a security venue. Security researchers submit to AI conferences; the reverse does not happen in this corpus.

One limit on the reading. A venue is coded only where the record carries one, so this covers 101 of 588 security-regime papers and 80 of 341 on the safety side. The rest are preprints with no venue recorded, and we draw no conclusion about where they will land.

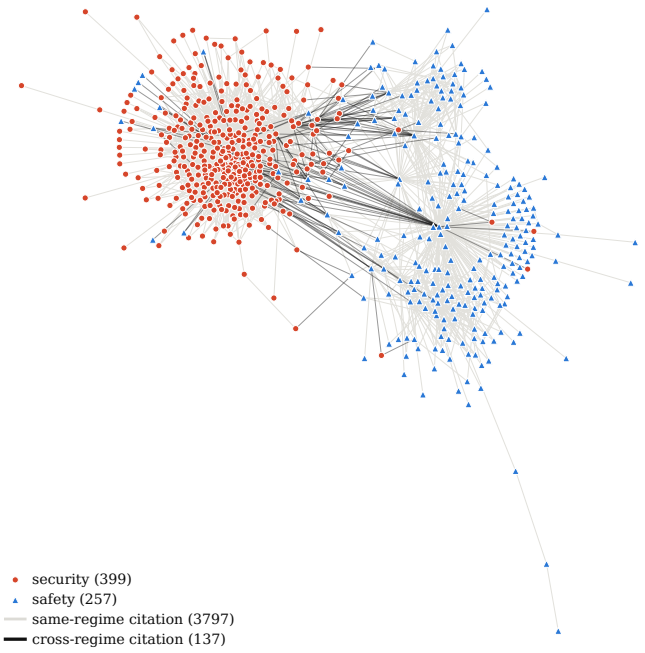


Fig. 5. The 709 papers that named a technique, as a citation graph. Security papers are red circles and safety papers are blue triangles, so the two regimes stay apart in a black-and-white print. Cross-regime edges are drawn dark against the pale same-regime ones. Papers with no resolved edge inside the corpus are omitted.

So the two bodies of work are not kept apart by their proceedings. They are kept apart by what they cite, which Section III-D measures. That is a stronger claim than a venue split would have been, and a harder one to explain away: these papers appear beside each other and still do not read each other.

D. Cross-citation between the literatures

Neither side reads the other. 11.0% of security-regime papers (47 of 426) cite a safety-regime paper, and 11.3% of safety-regime papers (32 of 283) cite a security one. About 89% of papers on each side were written without reference to the other community.

What carries the cross-regime citations makes the rate look better than the engagement is. 137 of the 3,934 resolved edges (3.5%) cross regimes, and they concentrate on a handful of papers either side can cite in an introduction without adopting anything from it. The four most-cited bridges are lost-in-the-middle [10], Greshake et al. [3], PoisonedRAG [4] and Self-RAG [34]. These are the papers that name the problem, and a citation to one places a paper in a setting rather than showing a method crossing the boundary. Figure 5 shows the shape, two clusters joined by a narrow band.

E. Recovered defense–attack pairs

A (defense, attack) pair means some paper evaluated that defense against that attack. The registry's unit is the claim: a method enters it when its paper proposes it against a named failure mechanism, not when the paper demonstrates that it works. *Defense* therefore means proposed-against, and the distance between proposing and demonstrating is part of what this section measures. We recovered pairs by four routes: reading defense papers, a citation-and-full-text recovery pass, resolving benchmark names to their constituent attacks where the paper named them, and a reverse scan of attack papers. The last route matters because it is the only one that can observe a defense *losing* to an attack published after it. The final matrix holds 349 confirmed pairs.

The matrix is sparse in a specific way. 215 of 541 defenses (39.7%) appear in at least one pair, and 145 of those (67.4%) appear in exactly one. The most cross-tested defense appears in 8 pairs, and 13 appear in four or more. But of the pairs belonging to those 13, 46% were contributed by attack papers rather than by the defenses' own evaluations; for the broadest case, 7 of its 8 pairs come from attack papers. Breadth here is not evidence a method was evaluated widely by its authors. It is evidence that attackers chose it as a target.

How much of this gap is the field and how much is our instrument. It is tempting to read the complement of the matrix as "never tested", and that reading is wrong. A defense evaluated on AgentDojo without naming which injection it faced was evaluated adversarially; it is simply not a named pair, because expanding a benchmark into its constituent attacks would manufacture coverage. So we classified every one of the 541 defenses in the registry by what its own paper reports in its evaluation section, asking the coarser question: was this method run against an adversary at all? 311 were (57.5%): 215 appear in a confirmed pair, 59 carry an unambiguous adversarial signal inside their own evaluation section, and 37 were run on an adversarial benchmark without naming the attack. 19 state that they ran no real comparison at all. 197 were evaluated with real baselines but never against an adversary, on capability and quality benchmarks instead. For the remaining 14 the coded fields do not say.

That first group is exactly the set the matrix matches, so the rest of this audit partitions the 326 fixes the matrix leaves empty, and it answers the question the empty space raises. 96 of them met an adversary whose attack carries no name the literature reuses: they were tested, and pairing on names cannot see it. 197 met no adversary at all. The 19 without a real comparison and the 14 that do not say are the remainder. The empty space is therefore mostly the field and partly this instrument, in a ratio the audit fixes rather than leaves open.

That 197 is two populations, and they read differently. 173 are safety-regime fixes, most of them against silent-corruption and reasoning-corruption. These are not untested methods: a fix for a stale or conflicting passage is demonstrated by accuracy rising on cases where the context is wrong, and there is no adversary to put in the experiment. Their behavior under attack is unmeasured rather than unsound, and they are the population this paper's campaign probes. The other 24 sit in the security regime, and 18 of them name an adversarial threat model in their own evaluation coding while putting no adversary in the experiment, most against goal-hijack. That is

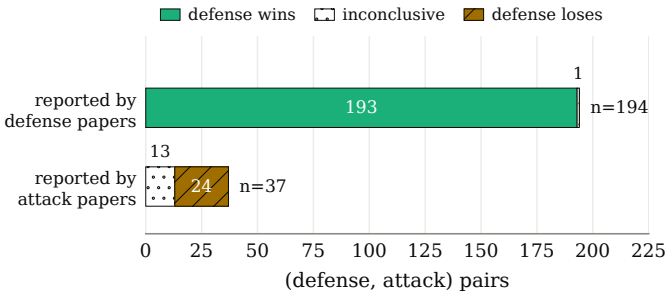


Fig. 6. The same defenses and the same attacks, reported by the two sides of the field. A defense paper reports its defense winning in 193 of 194 pairs. An attack paper reports the defense winning in none of 37.

a different failing from the first group. It is a defense whose evaluation does not meet its own stated threat model, and it is the one place in this audit where the gap is the paper's rather than the field's convention. The classification is conservative by construction — anything ambiguous lands in that last group rather than in a positive one.

Every count above is a positive finding: something the paper says, not something a pass failed to find. We report no number of the second kind, and the reason generalizes. We built three automated passes over these papers, and hand-checking each found the same asymmetry: what a pass flagged as present was reliable, and what it flagged as absent was about half wrong. A pattern can confirm that something is present in a paper; it cannot establish that something is absent, because absence of the phrasing it knows is not absence of the thing. A claim that something was never tested should state whether it means "not done" or "not resolved".

F. Reporter bias in recovered pairs

For every pair we also record which side of the field reported it and what outcome was reported. Figure 6 puts the two reporters side by side. Defense papers report their defense winning in 193 of 194 pairs, with the remaining 1 reporting no result we could resolve. Attack papers report a defense winning 0 times in 37: they report it losing in 24 and reach no directional verdict in 13. No pair has been reported by both sides: 0 of the 231 pairs carrying a reported outcome carry an account from a defense paper and an attack paper alike.

Neither number measures how often defenses work. The pair of numbers measures publication incentive, and the size of the gap is the reason the rest of this paper re-runs the defenses rather than citing either side's claim.

G. The untested region

The axes so far describe what each community studies. The question this paper turns on is narrower: *has either community ever tested its methods against the other's problems?*

A pair is one defense run against one failure mechanism by some paper. Sort every pair into four boxes — who built the defense, against whose problem it was run — over the 349 pairs whose two sides both sit in one regime.

Along the diagonal, nearly everything is there. Each community tests its own defenses against its own problems: 264 security-on-security pairs and 80 safety-on-safety. The two off-diagonal boxes are the ones that would answer our question, and they are almost empty. 1 security defense has ever been run against a safety failure mechanism; 4 safety methods have been run against security attacks. That is 5 pairs in 349, 1.4%, and the other 98.6% never leave their own community.

That near-empty off-diagonal is the untested region. It is not a claim that these methods fail outside their own regime. It is a claim that nobody has checked, and a deployed system does not sort its inputs into "an attacker wrote this" and "this is merely stale".

One reading would make that gap uninteresting: that the communities never tested across the boundary because they already knew the answer, so running the experiment adds nothing. Three measurements say otherwise. The communities do not read each other, and a judgement cannot be informed by work nobody cites: 11.0% of security-regime papers cite a safety-regime paper, and 0 of 231 reported pairs carry an account from both sides. And 18 defenses claim an adversarial threat model in their own evaluation coding while putting no adversary in the experiment, which is not what withholding a test you expect to fail looks like.

The single crossing is worth naming, because it is the strongest evidence against our own framing and it is one pair. TRACE [35] is a trajectory compressor for long-horizon agents whose own evaluation is adversarial, which is what places it in the security regime, and it is reported against context-length-induced degradation, a safety mechanism. It comes from a paper whose subject matter spans both communities, which is precisely the kind of paper that would cross first. One instance in 349 is not a literature that tests across the boundary; it is the exception that shows what crossing would look like.

This is consistent with the citation rates and sharper than them. The communities do not read each other, so they do not test each other, so the question of whether either one's methods survive the other's regime has no answer in the published record.

Nor is it answered by ordering. 50 of 349 pairs (14.3%) have a defense that predates the attack it was tested on, so defenses do meet attacks their authors could not have designed against. But only 1 pair is both earlier *and* cross-community, so meeting an unforeseen attack and meeting the other community's attack are not the same event. Figure 8 shows the full distribution.

H. Division of labor, not duplication

If two communities work one problem without reading each other, the second to arrive should rebuild what the first already has. The decisive evidence is not what the methods are called but where in the pipeline they act, which is a coded field rather than a reading of names. Security defends where the text enters: 104 of its defenses act at ingestion. Safety defends where the text is used: 139 of its defenses act at reasoning. Figure 3 draws that split against the pipeline and Figure 7

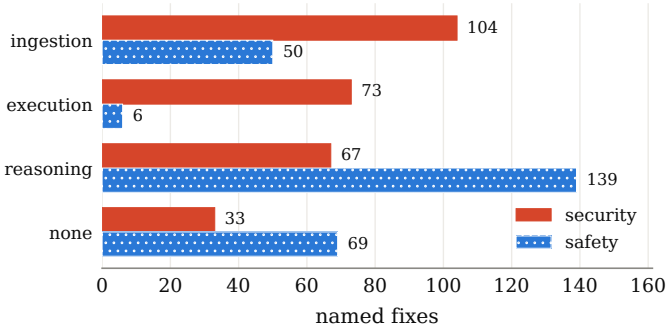


Fig. 7. Where each regime's fixes intervene. Security keeps the text out and contains what it does; safety improves reasoning over text already accepted. Points run in pipeline order. Safety carries a dotted fill and is the lower bar of every pair, so the two separate in print without colour.

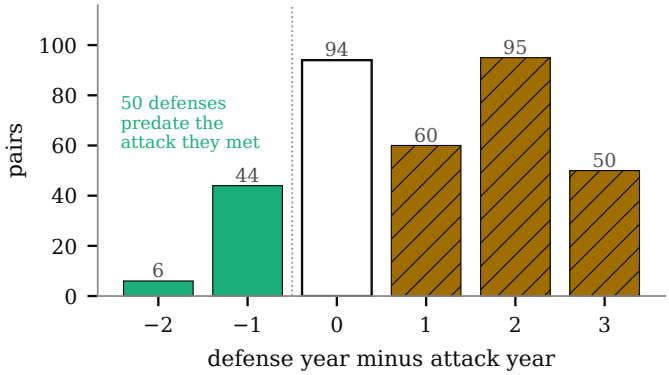


Fig. 8. Publication lag between a defense and the attack it was tested on.

counts it. Two bodies of work that intervene at different stages are not building the same thing, whatever they name it.

The names agree with the coding. Inside each taxonomy cell we looked for a security-regime name and a safety-regime name close enough to be the same thing relabelled: among 236 named mechanisms there are 0 such collisions, and among 541 defenses there is 1. That test can only catch relabelling, so it confirms the division rather than establishing it.

The two sides name different halves of one causal chain. 10 of our 40 occupied cells hold mechanisms from both regimes, 74 security and 34 safety. Read them side by side, and the security entries are overwhelmingly ways of getting bad text in, while the safety entries are reasons the model believes bad text once it is there. The RAG × silent-corruption cell is the clearest case: PoisonedRAG, BadRAG [36] and their relatives on one side; over-trust in conflicting external evidence, lost-in-the-middle, and knowledge-conflict circuits on the other. One side builds the attack. The other side has already explained why it works and is not cited for it.

The consequences each side names show the same division. Goal-hijacking, an attacker's objective, is 108 security mechanisms to 2 safety ones. Reasoning corruption, a degradation, runs the other way: 20 safety to 1 security.

That last reading is ours, taken from the mechanism names and their papers rather than from a coded field, and it is not clean at the edges. A few safety-regime entries name a delivery route rather than a susceptibility, KV-cache contamination and incidental prompt injection among them. The intervention points the subsection opens with are coded, and the argument rests on them.

If the two sides were duplicating each other, the cost would be wasted labor and the field would still end up holding two defenses that work. Dividing the problem costs something else: it leaves a chain with a gap in it. A reasoning-stage defense sits downstream of an adversary who already controls its input, and no amount of better reasoning recovers a document that was written to be believed. That is not a rhetorical point. It is the mechanism behind the sign flip in Section VI, where methods that shift trust toward the retrieved document help when the document is honest and hurt when it is written by someone else.

Publication dates say the same thing from a second direction, and they say which way the division runs. 7 of our nine channels have named mechanisms from both regimes, and we compare the first paper from each side in each of them. The security literature arrives first in 4 of 7. Both arrive in the same year in 3. The safety literature arrives first in 0. The median lag where security leads is 1 year and the longest is 3. The supply-chain channel, excluded here for having no safety-regime entry at all, is the limit case of the same pattern.

Set that beside the citation rates. A safety-regime paper naming a failure in a channel is, more often than not, working ground a security paper reached one to three years earlier, in a literature that only 11.3% of safety-regime papers cite at all. It is not renaming that paper's mechanism — the collision count above says it is not — and arriving second at a surface is not a fault. We are claiming the obvious consequence, which is that neither side can tell whether the other has already answered its question, and Section III-G shows that nobody has checked.

Two limits on this reading. The dates are publication dates in our corpus, so a channel that security reached first in work we did not include would move the count. And the direction may partly reflect that security publishes attacks, which are discrete and nameable, while the safety side publishes degradations, which are noticed before they are named.

Neither community can close the chain alone, and this section says why. The methods that keep bad text out and the methods that reason better about text already present are built by different people, published at different venues, and evaluated against different problems; no single method in the census covers both stages. A system that wants coverage at ingestion and at reasoning therefore has to compose methods from both literatures, and the census holds no paper that does. Whether such a composition would help is a separate experiment, and not one this paper runs: every method in Section VI is measured alone. What the census establishes is narrower and still unwelcome. The two halves exist, the stages they cover are complementary, and nobody has put them together or tested what happens when they are.

The boundary could be ours rather than the field's, and it is

not drawn by exclusion. We assign the regime label per paper, every paper takes one of the two, and papers on the boundary take the regime their own evaluation adopts. A paper that spans both communities therefore still sits among the 349 pairs; we set none aside. The single crossing above comes from one of them, which is what one would expect — a paper that reaches across is the kind of paper that crosses.

IV. The platform

Section III-G ends at a cell nobody has filled. This section describes the instrument we built to fill it. It is one harness. It runs a published defense against a published attack under one protocol, with the controls fixed before any run. The next two sections report what it produced, and together they answer RQ2.

A. Pair selection

The taxonomy picks the (defense, attack) pair under two conditions. The defense and the attack must occupy the same taxonomy cell, which means the content enters through the same channel and causes the same consequence. And no paper in the corpus may report having run them together. A defense that repairs quiet answer corruption arriving through retrieval ought to apply to an attack that arrives the same way and does the same damage. The taxonomy says so. Nobody had checked.

We are not proposing new defenses. Every method we run is published, and every attack we run is published. The contribution is the pairing.

B. The reconstruction standard

The taxonomy decides which pairs we run, and this rule decides what counts as running one. Every number we report comes from running a defense's own released code and weights. We never cite a paper's reported numbers as evidence about that paper's method. Where a harness had to be adapted to our data, we adapted the harness and left the method and the evaluation rubric untouched. Where an author's rubric and ours differed, we used the author's.

The standard cost us coverage. Several defenses could not be evaluated at all, for reasons that lie in what their artifacts ship rather than in what their methods do, and we record those as roadblocks rather than as failures of the method.

C. Pre-registered experimental rules

Released code is not enough on its own. A negative result is easy to produce by accident, so four rules guard against that, and we fixed all four before any run.

The control fires first. A defense that is broken in our hands tells us nothing about transfer, so before any transfer number we check that the method still does, in a clean context, what its authors say it does. The check is directional, and it is set per method, because no two papers claim the same quantity. ParamMute [13] reports a gain in context recall, not in accuracy. CK-PLUG [14] claims no accuracy gain at all: its contribution is a knob that moves memorization rate while accuracy is held, so its control fires on that rate instead. ACD [37] states its improvement without a figure. SCR's [38] own authors report it underperforming their baseline on Llama-3-8B, the model we run it on, so on this model there is no published gain to reproduce, and agreement with them means moving the same way they report. For each method, we therefore check the direction of the change, on the metric that paper uses for the model it used. We record which quantity each control was read on. Where no comparable published figure exists, we say so rather than manufacture a threshold.

The attack works first. If the attack does not degrade the undefended arm in the same run, the defended arm means nothing. Every result below is reported next to an undefended arm measured on the same items at the same doses with the same scorer.

Effects are gated on a minimum detectable change. For each run we compute the smallest difference detectable at the given sample size with $\alpha = .05$ and power .80. Differences below it are reported as "no resolvable effect", never as "no effect". This matters because several published claims in this area rest on differences that a run of their size cannot resolve.

Attacks are not adapted to the defenses they meet. An attack that is not built for a defense and defeats it anyway is a lower bound on attack strength: if the cheapest payload wins, an adapted one also wins. So non-adaptive evaluation is conservative for a claim that a defense fails, and invalid for a claim that one holds. Every failure we report is of the first kind. The single resistance result we report is of the second, and we test it adaptively rather than asserting it.

Predictions were written into the runner scripts and committed to version control *before* each run, and they ship with the artifact.

D. The two rigs

The retrieval cell pairs defenses built for knowledge conflict against attacks built to poison a retrieval corpus. The victim model is Llama-3-8B-Instruct [39] or Mistral-7B-Instruct [40], depending on what each defense released. The data is Co-ConflictQA [13] over Natural Questions, and open Natural Questions for the attacks that need a retrieval pool.

Those defenses turn the same dial. ACD decodes from the no-context distribution and pulls it toward the with-context one by a weight computed from entropy. CK-PLUG interpolates the two distributions under a single parameter. ParamMute suppresses the feed-forward layers that carry parametric memory, forcing the model onto the retrieved text. SpARE [41] and SHIFT [42] steer along a context-faithfulness direction in activation space. COIECD detects a conflict between the two sources and pushes toward the context. SCR makes the same trade through prompting rather than decoding. Each changes *how much* the model leans on retrieved text. None asks *which* retrieved text deserves trust.

The agent cell pairs ingestion-stage and execution-stage defense techniques against prompt-injection attacks inside the AgentDojo banking suite [24], with a Qwen2.5-14B-Instruct [43] agent.

TABLE I

ITEM SETS AND PAYLOADS, BY OWNER. NO ITEM SET IS OURS. THREE PAYLOADS COME FROM THE ATTACK LITERATURE AND FIVE ARE OURS, AND NONE OF THE FIVE IS A NEW ATTACK: THREE ARE EDITS OF A FIELD THE BENCHMARK ALREADY SHIPS, AND TWO REUSE THE ITEM SET'S OWN RETRIEVED PASSAGES, WHICH IS WHAT MAKES THEM CONTROLS RATHER THAN ATTACKS.

Item set	**Released by**
CoConflictQA [13] over Natural Questions	the ParamMute authors
Open Natural Questions [44], with its retrieval pool	the RobustRAG authors
FaithEval [45], SQuAD [46] negatives	the FaithfulRAG [47] authors
AgentDojo banking suite	the AgentDojo authors
Payload	**Origin**
Corpus poisoning	Zhong et al.'s released passage, unmodified
PoisonedRAG, BadRAG	The published constructions
False evidence	CoConflictQA's own parametric-evidence field
Distractor evidence	**ours**: one substitution into that field
Adaptive distractor	**ours**: the same claim, reshaped against one defense's rule
Gradient distractor	**ours**: that passage and a suffix optimized against the same rule
Stale document	**ours**: a real retrieved passage, gold answer swapped
Misretrieval	**ours**: a real passage, unmodified, retrieved for another question

Every run in this paper used one NVIDIA RTX A6000 with 48 GB, driver 535.274.02 and CUDA 12.8, under Python 3.10.12, PyTorch 2.10.0 and `transformers` 5.5.0. Two defenses cannot use that last version. ACD's released decoder imports symbols that `transformers` removed after 4.44, which its own README pins, so it runs in a separate dependency set. CK-PLUG ships its own 4.49 fork and its README says to install it. We record this because it is a result as much as a configuration detail. The defenses in a single cell of this taxonomy cannot share one Python environment. That is the reproducibility problem of Section VII, one layer further down. The released artifact builds each set and verifies it.

E. Datasets

An **item** is one question, its gold answer, and the context the model is given to answer it from. Every item in this paper belongs to the authors of the method or the benchmark we are running. Table I lists each source and whose it is. This is deliberate rather than convenient. A defense evaluated on the distribution it was designed for cannot blame its failure on an unfamiliar dataset. A result obtained that way is harder to dismiss than one obtained on data we chose ourselves.

Every comparison in this paper is between arms measured on the same items. Rates computed on different item sets are incomparable rather than merely noisy, so we checked this mechanically rather than by inspection: across every result registry, no run compares two arms measured on different items.

The one place the item set narrows is the distractor payload, and it narrows for both arms together. It is defined on 781 of CoConflictQA's 1409 items. An item qualifies on two conditions. Its remembered answer must appear verbatim in its own evidence sentence, so that the substitution needs no rewriting. And its question type and answer magnitude must admit at least three candidate distractors, so that the planted answer is a plausible example of its kind rather than an absurdity a reader would dismiss. Every arm in those runs, including the undefended and no-retrieval arms, is measured on that same subset.

F. Payload classes

The retrieval cell evaluates every item whose context is at most 2000 characters. The cut is on context length alone, never on an outcome, and it is applied once and then frozen, so every defense and every payload meets the same items. Its purpose is memory. Contrastive-decoding defenses hold two forward passes of the model at once, and this keeps the whole cell inside one card at a batch size every defense can share.

On those items we hold the dosing and the scorer fixed as well, and vary only the *form* of the planted passage. There are five forms. They differ in how the passage reads and in what it does to the evidence, and a defense that behaves differently across them tells us which property it is actually keying on.

Corpus poisoning [5], [48] optimizes a token sequence to be retrieved for many queries. The resulting passage is incoherent to a human reader. It attacks by displacing honest documents.

False evidence does the opposite. It plants a fluent, well-formed sentence that asserts the wrong answer the model itself would have given from memory. It is drawn from CoConflictQA and reads like an ordinary encyclopaedia sentence. It attacks by agreeing with the model's prior.

Distractor evidence is that same fluent sentence with one word changed, so that it asserts a third answer which is neither the truth nor the model's prior. It is not a stronger attack. It is a measuring instrument, and Section VI-B explains why it was necessary.

An adaptive distractor is the distractor payload again, in whichever surface form minimizes the quantity one defense's rule measures. A gradient distractor is that passage with a short optimized suffix appended against the same quantity (Section VI-D).

Three of the five are ours, the distractor and its two optimized forms, and all three are edits of a field that already ships with the benchmark. That field is worth drawing out. False evidence is the `parametric_evidence` field that ships inside CoConflictQA, the benchmark the ParamMute authors released themselves. One of the defenses we break is broken with a field from its own dataset.

Dose, and the one item set that has none. The **dose** is how many of the retrieved passages are planted. It is the axis every result in this paper is read along, because a defense that holds

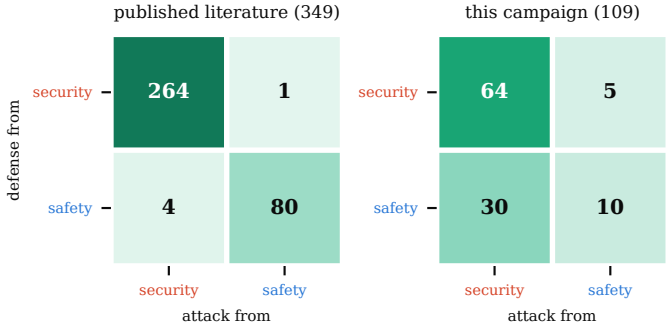


Fig. 9. Who built the defense (rows) against whose problem it was run (columns). Left, every confirmed pair in the literature; right, every pair in this campaign. Shade is each cell's share of its own panel, so the two panels are not on one colour scale and the printed count is the value. The literature crosses between the communities in 5 pairs of 349 (1.4%); one of its two off-diagonal boxes holds a single pair. All four boxes are occupied on the right.

against one poisoned passage and fails against five has not failed in the same way as one that never helped.

We take the dose from the attacks rather than define one. Their released constructor is parameterised by how many of the top k retrieved passages it replaces, where it places them and how often the planted sentence repeats inside its passage. We keep that construction and sweep the one parameter: j of the top ten, for j in $\{0, 1, 2, 3, 5, 10\}$. Their own default is $j = 1$, so their setting is one of our points and the rest are further readings of the same instrument.

One item set has no such axis and we do not give it one. CoConflictQA supplies a single context passage per item, which either carries the conflicting claim or does not, and a single passage has no ranked list to dose. We report it in the two conditions its own authors run, a clean context and a corrupted one. The two item sets are never combined in one number.

Every result file also carries a fingerprint of the payload it was produced with, so a run cannot be read under the wrong attack label.

V. Experiments

We executed 110 (defense, attack) pairs across 18 defense implementations and 595,240 scored episodes. Every method ran from its authors' released code and weights. Figure 9 sets the campaign beside the published literature.

The main experiment puts a defense from one literature in front of an attack from the other. We ran 36 such pairs, all on retrieval question answering over Natural Questions. 30 of them take a knowledge-conflict method, built for a document that is merely out of date, and attack it with corpus poisoning, PoisonedRAG or BadRAG. The remaining 6 run the other way: a defense built against an attacker meets a failure nobody wrote, such as a passage retrieved for the wrong question or a real passage whose answer has since changed.

A second experiment pairs each method with the kind of attack its own authors used. These 64 security pairs and 10 safety pairs tell us nothing about whether a method carries across, and we ran them so that a failure in the paragraph above cannot be blamed on our harness. Most of them are the agent experiment: 56 pairs in which eight prompt-injection defenses meet seven injected-instruction attacks inside the AgentDojo banking suite [24].

A third experiment holds the defense and the dose fixed and changes only the shape of the planted passage. The five shapes are a token sequence optimized to be retrieved, a fluent sentence asserting the answer the model would have given from memory, the same sentence asserting a third answer instead, a real passage retrieved for another question, and a real passage with its answer swapped. A defense that behaves differently across them is keying on the shape of the text rather than on the attack it was named for.

A fourth set of runs supplies the measurements published evaluations leave out. Every run above carries three arms on the same items: a clean arm with nothing planted, an undefended arm, and an arm that sees no document at all. We also re-ran the retrieval experiment on a second base model, and we ran both released implementations of the one defense design that held, which share a principle and no code downstream of it.

The last two experiments test the explanation rather than the methods. We built two attacks against the single defense that resisted, one searching over surface forms and one optimizing a suffix with gradients, both targeting the quantity that defense's own rule measures. And we stacked two defenses that act at different stages of the same pipeline, to test the design rule the observations below arrive at.

VI. Observations

Observation 1. Planting a false claim gives an attacker a measurable advantage over an honest mistake. Pushing the right evidence out gives no advantage at all. The research questions below assume that a planted passage and an honest mistake do comparable damage, so we measured that first, with no defense in place. Table II runs five kinds of planted passage on the same items, with the same number of poisoned passages in the same positions, scored the same way.

Where the passage plants a false answer, writing it to persuade helps the attacker. With one poisoned passage in ten, the model gives the planted answer 20.6% of the time for PoisonedRAG's crafted passage and 8.7% for a real passage whose answer has changed. The gap never falls below 9.9 points, which is above the 9.9-point detection threshold, and at five poisoned passages the crafted passage also costs more accuracy, 63.8% against 70.5%. The honest mistake still does real damage: it moves the model to the false answer 14.9% of the time, against 2.7% with nothing planted. Repeating the crafted passage five times, as the published attack does, changes these rates by at most 3.0 points.

Where the passage only pushes the right evidence out of the retrieved set, the attacker gains nothing. Zhong et al.'s optimized passage and a real passage retrieved for another question give the same results: 2.2% and 3.2% planted answers, 68.2% and 73.2% accuracy, with every gap below the threshold at

every number of poisoned passages. For this kind of attack, an attack and an ordinary retrieval error are the same event.

A. RQ2: Does a defense built for one literature's problem survive the other's?

Observation 2. The methods built for honest error all decide how far to trust the retrieved document over what the model already knows. Trusting the document more is right when the document is out of date, and wrong when an attacker wrote it. Seven of the ten methods we ran on retrieved documents work this way: ACD, CK-PLUG, COIECD, ParamMute, SCR, SpARE and FaithfulRAG. None of them can tell a real document from a planted one, because both are the document. When the document is honest and newer than the model's memory, leaning on it helps. When an attacker wrote it, leaning on it hands the attacker the answer.

Table III shows the sign change. SpARE gains 14.2 points on a clean context and loses -10.2 on a corrupted one, both above its threshold. ParamMute gains 9.3 points on a clean context and ends 4.2 points below the undefended model on a corrupted one. CK-PLUG goes furthest. Under BadRAG's tone attack, which tries to make the model answer in a negative tone, the undefended model takes on that tone at most 4.0% of the time. With CK-PLUG it takes it on 78.2% of the time with one poisoned passage in ten and 96.0% with five. The defense does not fail to stop the attack; it carries it.

The other three methods are why this is a claim about a design and not about all defenses. DyPRAG turns the document into model weights, so it has no trust setting to turn. The two RobustRAG implementations answer from each passage separately and take a vote, which asks which passages agree rather than how far to trust passages in general.

Figure 10 shows the same methods as the number of poisoned passages rises. CK-PLUG and DyPRAG sit more than twenty points below the undefended model before anything is planted and stay there. The other four stay within a few points of it at every number of poisoned passages.

Observation 3. Suppressing the model's memory hurts only when the memory had the right answer. Under BadRAG's refusal attack, which makes the model refuse rather than answer wrongly, ParamMute protects strongly. With one poisoned passage in ten the undefended model refuses 52.4% of the time and answers 27.0% correctly. ParamMute cuts refusals to 0.2% and restores accuracy to 57.4%, within 2.4 points of its clean score. The mechanism is the same as in Observation 2. Suppressing memory removes a right answer when memory had one, and removes the conflict behind the refusal when it did not.

Observation 4. Across every defense and every attack on open Natural Questions, almost nothing helps. Table IV reads each pair against its own undefended model at five poisoned passages in ten. Of the 38 pairs, 3 help by more than their threshold, 10 hurt by more than their threshold, and 25 show no measurable difference. All 3 gains are against the refusal attack, where the method helps by getting the model to answer again rather than by resisting anything. Every measurable

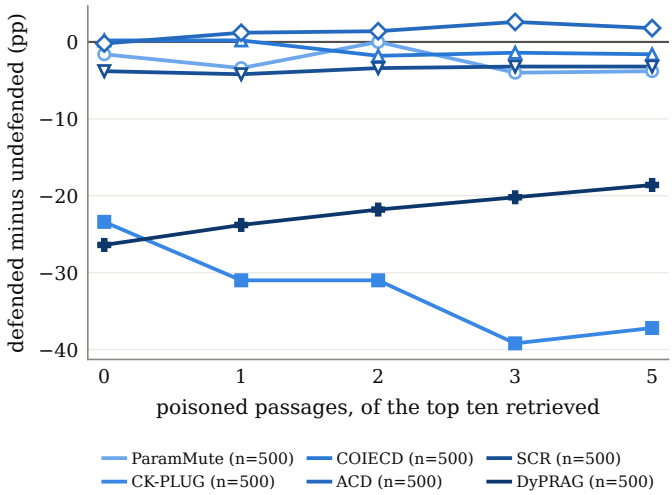


Fig. 10. Each safety-literature defense against corpus poisoning, by number of poisoned passages among the ten retrieved, measured as the defended model minus the undefended model in the same run. Zero means the defense changed nothing. Filled markers are resolvable at $p < .05$ by Fisher's exact test [49]; hollow markers are not. CK-PLUG and DyPRAG sit more than twenty points below zero before any passage is planted and stay there. The other four stay within a few points of zero.

result against an attack that plants a false answer is a loss. CK-PLUG and DyPRAG account for 7 of the 10 losses, and they are also the only two methods that cost accuracy with nothing planted, more than twenty points each. The other 30 of 38 pairs cost nothing measurable on a clean context.

Observation 5. Even with nine of ten passages honest, almost nothing helps. One poisoned passage in ten is the default of the released attack and the setting most favorable to a defense, because it leaves the most honest evidence to work with. 2 of the 38 pairs help there, and both are again the refusal attack. We report the worst result for each defense rather than the average, since a defense beaten by one of three attacks is not two-thirds secure.

Observation 6. Defenses built against an attacker help against an honest mistake only when the mistake plants a false claim. We ran RobustRAG, in both its released implementations, against four failures that no attacker wrote.

Against a real passage whose answer has changed, RobustRAG helps with one or two such passages and then turns harmful. With one, the model gives the outdated answer 13.4% of the time undefended and 3.0% and 3.7% with the two implementations. With five, the defended model gives it more often than the undefended one, 49.4% and 54.6% against 34.2%, and answers correctly less often, 37.2% and 21.3% against 54.3%. A passage asserting a third, wrong answer shows the same pattern: 2.0% and 6.0% against 25.7% undefended with two such passages, but 27.7% and 40.0% against 36.3% with five. Against a passage retrieved for the wrong question, which plants nothing, it never helps: both implementations start below the undefended model, 52.3% and 54.0% against 58.7%, and are still below it with five such passages.

TABLE II

DOES IT TAKE AN ADVERSARY? FIVE PAYLOADS ON THE SAME 403 ITEMS, DOSED AS j OF THE TOP TEN RETRIEVED PASSAGES. THE TWO POISONEDRAG ROWS AND THE STALE DOCUMENT ASSERT THE *SAME* FALSE ANSWER IN THE SAME SLOTS, SO THE ONLY VARIABLE BETWEEN THEM IS WHETHER THE SENTENCE WAS WRITTEN TO BE BELIEVED. MISRETRIEVAL IS A REAL, UNMODIFIED PASSAGE RETRIEVED FOR ANOTHER QUESTION. LEFT: HOW OFTEN THE MODEL GIVES THE PLANTED ANSWER. RIGHT: ACCURACY ON THE GOLD ANSWER. THRESHOLD 9.9 PP AT THIS SAMPLE SIZE; THE UNDEFENDED MODEL IS THE ONLY ARM, SO NO DEFENSE IS INVOLVED.

		adopts the planted answer (%)					accuracy on the gold answer (%)				
payload	provenance	0	1	2	3	5	0	1	2	3	5
PoisonedRAG, repeated $\times 5$	adversarial	2.7	21.8	25.3	27.1	34.7	73.2	66.2	64.0	64.5	62.0
PoisonedRAG, placed once	adversarial	2.7	20.6	24.3	26.1	31.8	73.2	68.5	67.0	65.8	63.8
stale document	incidental	2.7	8.7	14.4	12.9	14.9	73.2	70.7	71.0	71.2	70.5
misretrieval	incidental	2.7	2.7	2.7	3.0	3.2	73.2	74.2	73.0	72.2	73.2
corpus poisoning	adversarial	2.7	3.0	2.5	1.7	2.2	73.2	70.2	68.5	69.5	68.2
no retrieval	—	4.7					49.6				

TABLE III

DEFENSES THAT DECIDE HOW FAR TO TRUST THE DOCUMENT, AGAINST CORPUS POISONING, IN PERCENTAGE POINTS AGAINST EACH DEFENSE'S OWN UNDEFENDED ARM IN THE SAME RUN, WITH THE UNDEFENDED AND DEFENDED RATES IN BRACKETS. "CLEAN" IS A CONTEXT WITH NOTHING PLANTED, WHERE EACH DEFENSE SHOULD SHOW ITS OWN PUBLISHED BENEFIT. "CORRUPTED" IS A CONTEXT REPLACED BY THE PLANTED PASSAGE. BOLD CLEARS THAT RUN'S DETECTION THRESHOLD. RATES ARE ROUNDED FOR DISPLAY AND DIFFERENCES ARE COMPUTED BEFORE ROUNDING.

Defense	n	**threshold**	**Clean**	**Corrupted**
ParamMute	1409	5.3	**+9.3** (48.4 → 57.7)	**-4.2** (5.3 → 1.1)
CK-PLUG	800	7.0	**-10.1**† (25.1 → 15.0)	**-9.9** (10.1 → 0.3)
SpARE	500	8.9	**+14.2** (39.6 → 53.8)	**-10.2** (12.0 → 1.8)
COIECD	400	9.9	-0.8 (44.0 → 43.3)	-1.8 (3.8 → 2.0)

†CK-PLUG's control fires on memorization rate rather than accuracy, so its clean cell reports that metric.

Against attacks from its own literature, RobustRAG stops PoisonedRAG, as its own paper reports, by 11.0 points. Against corpus poisoning it neither helps nor hurts: 55.2% against 57.8% with one displaced passage and 50.2% against 54.0% with five, both inside the 8.9-point threshold.

Inside the tool-using agent, 24 of 56 defense and attack combinations drive the attack's success to zero, and every one of them acts before the agent reads the injected text or before it acts on it. None of the 7 combinations that act while the model reasons holds. Where in the pipeline a defense acts predicts whether it survives better than which defense it is.

SCR is the only method we ran that is designed with an attacker in mind. It shows the model its own answer without the document, the document, and the answer the document supports, and asks it to judge. It gains 2.8 points on a corrupted context and costs 13.5 on a clean one, roughly five times what it gains.

Answer to RQ2. No. Methods built for honest error lose to an attacker in proportion to how far they trust the document. Methods built against an attacker help with an honest mistake only briefly, and only when it plants a false claim.

B. RQ3: Is failure a property of individual methods or of a design choice they share?

Observation 7. What a defense does depends on the form of the planted passage, not on which attack it came from. ACD is the only method that both uses a clean document and resists corpus poisoning: it ends 9.3 points above the undefended model on a corrupted context. Corpus poisoning, though, was optimized to fool a retriever, not ACD, so this shows only that ACD resists that one passage. We then planted a fluent false sentence instead, holding everything else fixed. Table V shows the protection disappear. ACD keys on how uncertain a passage makes the model. An optimized token string is incoherent and raises uncertainty, so ACD discounts it. A fluent false sentence lowers uncertainty, so ACD trusts it more, and the defended model ends below a model given no document at all.

Three more methods show the same dependence on form (Table VI). ParamMute is harmful against both forms, by 13.5 and 12.3 points, but how well it suppresses the model's wrong memory falls from -42.5 to -8.8 points. Under corpus poisoning the wrong answer lives only in memory, so suppressing memory removes it. Under false evidence it lives in memory and in the planted passage, and ParamMute reaches only the first. CK-PLUG's cost grows from -4.5 to -9.9 points under corpus poisoning and stays near -4.4 under false evidence. COIECD changes nothing measurable against either.

We do not rank the two forms. Before running them we wrote down the result that would contradict our explanation: a defense doing better against false evidence than against corpus poisoning. CK-PLUG does. But false evidence is roughly twice as damaging to the undefended model, which holds 7.3% against 10.1% for CK-PLUG and 1.8% against 3.8% for COIECD, so there is less left for a defense to lose. A comparison across forms measures the attack's strength as much as the defense, so we compare only within a form.

Observation 8. No defense moves any answers toward the real document. When the planted passage repeats what the model already believes, a correct answer from memory and a copied answer look the same. We therefore built a passage asserting a third answer, neither the true one nor the model's own, by changing one word in the item's own evidence sentence. An output can then be read three ways: the true answer means the model read the real document, its own answer means it recalled from memory, and the third answer means it copied the planted passage. With nothing planted the third answer appears 0.8% of the time, and 1.0% of the time

TABLE IV

EVERY DEFENSE AGAINST EVERY ATTACK IT HAS MET ON OPEN NATURAL QUESTIONS, DOSED AS j OF THE TOP TEN RETRIEVED PASSAGES. EACH CELL IS PERCENTAGE POINTS AGAINST THAT RUN'S OWN UNDEFENDED ARM ON THE SAME ITEMS: *CLEAN* IS DOSE 0, WHAT THE DEFENSE COSTS WHEN NOTHING IS PLANTED, AND *5/10* IS FIVE POISONED PASSAGES OF TEN, THE LARGEST CORRUPTION SIZE THESE ATTACKS' AND DEFENSES' OWN AUTHORS EVALUATE. A DASH IS A PAIR WE HAVE NOT RUN. THIS ITEM SET IS NOT CONFLICTQA AND THE TWO SETS OF NUMBERS ARE NOT COMPARABLE.

	PoisonedRAG		corpus poisoning		BadRAG DoS		BadRAG sentiment		distractor		misretrieval	
defense	clean	5/10	clean	5/10	clean	5/10	clean	5/10	clean	5/10	clean	5/10
ACD	−0.2	+1.4	−0.2	+1.8	−0.2	+35.8	−0.2	+0.0	–	–	–	–
CK-PLUG	−23.4	−22.2	−23.4	−37.2	−23.4	−29.4	−23.4	−44.2	–	–	–	–
COIECD	+0.2	−1.2	+0.2	−1.6	+0.2	+0.0	+0.2	+0.2	–	–	–	–
DyPRAG	−26.4	−18.0	−26.4	−18.6	−26.4	−1.0	−26.4	−21.2	–	–	–	–
ParamMute	−1.6	−5.6	−1.6	−3.8	−1.6	+28.0	−1.6	−3.2	–	–	–	–
SCR	−2.6	−2.8	−3.8	−3.2	−2.2	+18.6	−2.8	−2.2	–	–	–	–
RobustRAG (keyword)	−5.6	+5.0	−5.6	−5.0	−5.6	+2.6	−5.6	−3.2	−7.0	+3.7	−6.3	−2.7
RobustRAG (decoding)	−5.2	+4.4	−5.2	−8.4	−5.2	−10.8	−5.2	−4.4	−5.7	−7.3	−4.7	−3.3

TABLE V

ACD AGAINST TWO PAYLOAD CLASSES, $n = 400$, MDR 9.9 PP. THE DEFENSE IS UNCHANGED. ONLY THE FORM OF THE PLANTED PASSAGE DIFFERS. p IS TWO-SIDED FISHER ON THE ARM COUNTS.

Comparison, corrupted context	**Corpus poisoning**	**False evidence**
vs. undefended	**+9.3** (p<0.0001)	+3.0 (p=0.0593)
vs. no retrieval	+0.5 (p=0.907)	**-4.0** (p=0.0499)

TABLE VI

THE SAME THREE DEFENSES AGAINST TWO PAYLOAD CLASSES. EXCESS DAMAGE IS THE DEFENDED ARM'S CLEAN-TO-CORRUPTED COLLAPSE MINUS THE SAME COLLAPSE IN ITS OWN UNDEFENDED ARM; POSITIVE MEANS THE DEFENSE LOST MORE ACCURACY THAN DOING NOTHING WOULD HAVE.

Defense	n	**Corpus poisoning**	**False evidence**
ParamMute	1409	**+13.5**	**+12.3**
CK-PLUG	800	**+5.4**	+0.4
COIECD	400	+1.0	-1.3

from a model given no document.

ParamMute does its job and the attacker collects the result. It cuts answers from memory from 39.1% to 2.9%, and copying rises from 43.9% to 81.4% almost one for one, while correct answers stay below the threshold. ACD does the reverse. It cuts copying from 76.0% to 40.8%, the largest defensive effect in the campaign, and the answers it refuses to copy go back to memory, not to the real document. CK-PLUG copies 86.7% against its undefended model's 47.9% (Table VII).

The two directions are one mechanism. Every one of these methods chooses between the document and the model's memory, and the attack is inside the document. Turning the setting toward the document gives answers to the attacker; turning it toward memory gives them back to memory. Neither setting reaches the real document, because the choice they make cannot separate a real document from a planted one. Methods that turn toward memory also lose a share of answers to neither outcome: 16.8, 16.5 and 23.5 points for ACD, SCR and SpARE.

Answer to RQ3. A shared design choice. The methods fail in the same way because they make the same choice, and that choice does not contain the distinction an attacker exploits.

C. RQ4: Do the measurements the field leaves out change the result?

Observation 9. Without a no-document score, a defense that ignores the document looks immune to the attack. DyPRAG holds 5.8% on a clean context and 6.0% on a corrupted one while the undefended model falls from 63.8% to 2.5%. Read alone, that is the strongest defensive result in the campaign. The same model with no document at all scores 7.5%, which is not measurably different. DyPRAG is unaffected because it is not reading the document, and only a no-document score shows this. No paper in this family reports one.

Observation 10. Where two implementations of one design agree, the result belongs to the design. RobustRAG ships two implementations that share the idea of answering from each passage and voting, and share no code after that. Under BadRAG's tone attack both stay at or below 0.3%, across both implementations and two base models, where CK-PLUG on the same attack reaches 96.0%. Answering from each passage and voting does not open this hole; trusting the document does. Under the refusal attack neither implementation changes the result measurably on either base model, within the range the attack authors evaluate.

Answer to RQ4. One omitted measurement reverses a result: a no-document score turns the campaign's apparently strongest defense into one that is not reading its input. Two further checks, a second implementation and a second base model, leave the results inside the evaluated range unchanged.

D. RQ5: Does that explanation survive an adaptive attacker?

Observation 11. The one method that resisted loses more than half of its protection to an attacker who reads its rule. Every attack above was built without knowledge of the defense it met, which is a fair test when a defense fails and no test at all when it holds. ACD was our only holding result, so we attacked it directly.

ACD blends the model's answer with and without the document, weighted by how uncertain each one is. The attacker controls only the document, so the attack has one goal: make the document look certain. This also sets a limit. A perfect

TABLE VII

CONTEXT-RELIANCE DEFENSES AGAINST THE DISTRACTOR PAYLOAD, AT SATURATION, IN PERCENTAGE POINTS AGAINST EACH RUN'S OWN UNDEFENDED ARM, WITH THE UNDEFENDED AND DEFENDED RATES IN BRACKETS. MEMORY IS THE PARAMETRIC-ANSWER RATE, COPIED IS THE PLANTED ANSWER, CORRECT IS THE GOLD ANSWER; THE THREE ARE DISJOINT, SO THE RESIDUAL IS MASS THAT LEAVES THE PARTITION ALTOGETHER. $*$ CLEARS BOTH THE RUN'S DETECTABILITY THRESHOLD AND FISHER AT $p < 0.05$. RATES ARE ROUNDED FOR DISPLAY AND DELTAS ARE COMPUTED BEFORE ROUNDING.

defense	n	memory	copied	correct	residual
ParamMute	781	-36.1^* (39.0→2.9)	$+37.5^*$ (43.9→81.4)	-3.8 (5.4→1.5)	$+1.4$
CK-PLUG	781	-31.6^* (36.1→4.5)	$+38.8^*$ (47.9→86.7)	-7.2^* (9.3→2.2)	$+7.2$
COIECD	400	-5.5 (18.5→13.0)	$+4.8$ (39.0→43.8)	-0.8 (2.0→1.2)	-0.8
ACD	686	$+18.4^*$ (14.1→32.5)	-35.1^* (76.0→40.8)	$+2.6$ (2.3→5.0)	-16.8
SCR	400	$+34.8^*$ (14.0→48.8)	-51.2^* (66.5→15.2)	$+2.8$ (2.0→4.8)	-16.5
SpARE†	400	$+33.0^*$ (0.2→33.2)	-56.5^* (85.2→28.8)	$+6.8$ (0.8→7.5)	-23.5

†SpARE's run carries no undefended arm, so its row compares steering toward memory against steering toward the context. It is not a defended-minus-undefended delta and does not belong in the same column of judgement as the rows above.

TABLE VIII

ACD AGAINST THE DISTRACTOR PAYLOAD IN ITS PLAIN FORM AND IN TWO FORMS THAT MINIMIZE ACD'S OWN ENTROPY TERM, ON A CORRUPTED CONTEXT, $n = 686$. BOTH ADAPTIVE PAYLOADS ARE STRONGER AGAINST THE UNDEFENDED ARM TOO, SO THE DEFENSIVE GAP IS THE QUANTITY THAT ISOLATES THE DEFENSE. ALL THREE COLUMNS WERE SCORED IN ONE ENVIRONMENT; SEE SECTION VIII.

corrupted context	plain	best-of-5	gradient
median H_{ctx}	1.348	0.783	**0.614**
median d_α	0.525	0.653	**0.708**
undefended copies attacker	76.0	89.1	87.8
ACD copies attacker	40.8	64.0	**73.0**
defensive gap	**35.1**	**25.1**	**14.7**
protection removed	—	**29%**	**58%**
ACD recalls from memory	32.5	22.2	18.1
ACD answers correctly	5.0	3.2	2.6

attack of this kind turns ACD into the undefended model and cannot push it below that. We committed to that limit, and five further predictions, before building the attack.

We built two attacks. The first tries five phrasings of the same false claim for each item and keeps the one ACD finds most certain. The second appends a short suffix found by gradient search against the same goal. Both keep the false answer fixed, so any change comes from the form alone.

All six predictions held (Table VIII). The fair measure is ACD's margin over the undefended model, because both attacks also hurt the undefended model. The rephrasing attack removes 29% of that margin. The gradient attack removes 58%, while doing less damage to the undefended model than the rephrasing attack, so it is attacking ACD's rule specifically. With nothing planted ACD and the undefended model differ by at most 0.4 points under all three passages, so the attacks act only through the planted text. ACD degrades but does not collapse: by the bands we set in advance it keeps some protection, and it never falls below the undefended model.

Answer to RQ5. The explanation survives, and the one exception shrinks. The method that resisted did so against attacks not built for it, and loses most of that resistance to one that is. Every published evaluation of these methods in our corpus uses attacks not built for the defense.

VII. RECOMMENDATIONS

For the mechanism that removes evidence rather than planting a lie, the campaign finds no difference between an optimized passage and an ordinary retrieval error. The citation gap of Section III is therefore not two communities working on different problems, at least where that mechanism is concerned, and the recommendations below follow from treating it as one problem.

A. Controls both communities should report

Section VI-C reports one result that reads the other way without a no-document score, and Section VI-B one that reads the other way when only accuracy is reported. Neither measurement is expensive and neither is standard. We propose three reporting requirements.

Report a no-retrieval arm. Any defense that modifies how a model uses retrieved context must report what the same model scores with no retrieved context at all. Without it, a defense that has stopped reading the document is indistinguishable from a defense that has learned to resist a poisoned one. This is one extra arm and it costs one extra pass over the data.

Report where the answers went, not only accuracy. Under a planted third answer, ParamMute moves accuracy by less than its threshold while the share of answers copied from the attacker rises by 37.5 points. Accuracy alone reports that defense as harmless. Classifying each output as the true answer, the model's own answer, the planted answer or a refusal costs nothing beyond reading the generations already produced.

Report at least two base models. Our second-model check left the results unchanged within the range the attack authors evaluate, but it covers one defense design. A result on one model cannot show that it holds on another, so a single-model result should be read as specific to that model.

B. Conventions for measurable transfer

The controls above are about one experiment. These five are about what a paper says it covers.

State the regime the evaluation actually covers. Our defense census codes each method by what its own paper

evaluated, and 97% cover one regime. Many of those papers describe their contribution in terms broad enough to imply the other. One required sentence would close this. Name the regime the experiments cover. A reader of the paper, and the maintainer of a system, then sees the gap without reading the appendix.

Gate claims on a detectability threshold. Reporting a difference without the smallest difference the experiment could have detected makes "no effect" and "we could not tell" look identical. We computed this for every run and several of our own results moved from the first category to the second.

Say whether the attack was adapted to the defense. A defense that holds against an attack built for a different target has been shown to resist that attack and nothing more. Every published evaluation of a context-reliance defense in our corpus is non-adaptive, and when we built an adaptive payload against the one method that had resisted, 58% of its protection went (Section VI-D). One sentence naming whether the attacker knew the defense tells a reader which kind of claim they are reading.

Compose across the boundary, and measure the composition. Section III-H shows the two communities defending different stages of one pipeline, and Section VI shows the reasoning stage failing on its own once an adversary writes the input. The obvious next move is to put an ingestion-stage filter in front of a reasoning-stage method and see whether the pair holds where each half does not. We do not know that it works. Nothing in this paper tests a composed defense, and a filter that passes a fluent falsehood leaves the reasoning stage exactly where our results find it. The experiment is cheap, both halves are published with released code, and no paper in 929 has run it.

Test a fix for accidental failure against a deliberate one. A method aimed at stale or conflicting retrieval is evaluated on average-case benchmarks, which is the right choice for the problem it was built for and says nothing about the worst case. The cost of not checking is not a missing result: it is that the method ships with a benefit its users will read as general, and the worst case is where it reverses. Running one published poisoning attack against an existing evaluation is a day's work and needs no new data. A method that cannot be tested that way should say which regime its benefit holds in.

C. Artifacts

The reconstruction standard made artifact quality measurable as a side effect. Among the best-cited untested defenses in the cell we studied, two name empty repositories. Two more ship code without the trained weights it needs. One runs. Two further execution-stage defenses do not run reliably on any freely-available open-weight model. One more, FaithfulRAG, needs structured-output support that open weights do not provide in its harness. The ablated variant that does run shows none of its own benefit, so we report it as not reportable rather than as defeated.

We note this without blame. Releasing a working artifact is harder than it looks and is rarely rewarded. But a defense nobody else can run is a defense nobody else will test, so part of the coverage gap this paper measures is an artifact gap.

Only part of it. 326 of 541 fixes never appear in a confirmed pair, and Section III separates that number into the two things it holds. 96 were run against an adversary whose attack carries no name the literature reuses, which pairing cannot see. 197 were never run against an adversary at all. The second group is the larger one, and it is the field rather than the measurement. The fixes left over are validated both ways, or carry no record of what they were validated against.

D. Implications for deployment

Three practical points follow from the results.

A method that tunes how much the model trusts retrieved text is correct when the text is merely stale and dangerous when an attacker owns it. If the corpus an agent retrieves from is one an outsider can write to, the second case is the deployed case.

Consider a system that uses a knowledge-conflict method to improve answer quality and retrieves from such a corpus. The method operates outside the regime its authors validated, and our measurements say it is more likely to increase damage than to reduce it: ParamMute is worth +9.3 pp on a clean context and -4.2 pp once the context is corrupted, below the undefended model on the same items. The claim is about a design choice rather than a community. A method that converts the document to weights has no dial to turn and does not show the reversal, and one method in the family resisted until an adaptive attacker priced it.

A defense that holds against an optimized adversarial passage may fail against a fluent sentence that simply asserts a plausible falsehood. In our measurements no defense resisted both. The fluent attack is cheaper to produce. It needs no gradient access and no surrogate retriever, and it does roughly twice the damage to an undefended model at every dose. Evaluations built only around optimized payloads overstate protection. Do not call a defense tested on one payload class robust to context poisoning.

Finally, the channel matters more than the method name. Supply-chain, cross-modal and tool-metadata have fewer than one named defense per named mechanism. If an agent accepts third-party components or machine-readable tool descriptions, almost nothing published defends that route.

VIII. Threats to validity

Extraction error. We produced the corpus coding with an LLM-assisted extraction pass over 929 papers and 20 coded fields. Every count in Sections III-B to III-E depends on it. A human validation study is drawn and stratified by the coding's own confidence: 125 papers, 18 fields each, designed for a 6.2 pp confidence interval on overall accuracy. It has not been coded yet. We therefore report no measured extraction-error rate, and the counts in those sections should be read as unvalidated. The study is single-coded by decision of the project lead. When it returns it will bound extraction error

without separating a coder's mistake from the extraction's, so it will carry no inter-rater agreement and we will claim none.

Regime assignment. The cross-citation rates depend on classifying every cited paper into a regime. That classification is automated over the full discovery pool. A systematic bias in it would move the rates. We regard the direction of the finding as safe, because the gap between 11.0% citation and 98.6% same-regime evaluation is far larger than any plausible classification error, but the exact rates should be read as estimates. Two further limits bound them. The denominator is the 709 papers that named a technique rather than every screened-in paper, because a survey cites widely and names nothing, so counting it would measure reading habits. And only citations to papers inside that set register, so a paper reaching across to work we did not include does not count: every cross-citation rate in this paper is a floor.

The reverse direction rests on one defense. Most cross-regime pairs in the campaign take a safety-regime method and attack it. The other direction holds one family: RobustRAG against distractor evidence and against misretrieval, in Section VI-A. That is two mechanisms, one defense, one model and one dataset. The split result there — it holds against the planted claim and costs accuracy against the retrieval error — reads two cells, not a population. The other security defenses are absent because their released code does not run on our retrieval data, and nothing here says whether they would divide the same way.

Registry granularity. What counts as one named mechanism is a judgement. "Indirect prompt injection" is one registry entry and absorbs 21.8% of all tested pairs. 34 of the 64 named injection variants have no tested pair at all. A coarser registry would make coverage look better and a finer one would make it look worse. We chose the granularity the papers themselves use, which is the name each paper gives its own contribution.

Who-reports-what is observational. Section III-F reports a strong association between authorship and outcome. It is not a controlled comparison. The two sides largely write about different pairs. The association therefore reflects which pair each side chose to run, rather than differing conduct on the same pair. That selection is our interpretation.

The campaign is a sample, not a census. We ran 110 pairs out of a space of thousands. They were selected by the taxonomy rather than at random, biased toward pairs where both artifacts run. Defenses whose artifacts do not run are absent, and nothing here says whether they transfer.

Attacks are non-adaptive, with one exception. This is the criticism that invalidated DEEPSEC's defense rankings [50], and we inherit it. An attack that was not built for a defense and defeats it anyway bounds attack strength from below, so our failure results are conservative. A resistance result drawn the same way is not evidence of robustness at all. Section VI-D tests the one resistance result we report. Every other defense in the campaign would need the same treatment before any claim that it holds.

Variance. Carlini lists "no variance reported" as one of five fatal flaws in DeepSec [50]. His own check found a standard deviation of 6.8 points on a reported attack success rate. That is enough to make many of DeepSec's pairwise comparisons meaningless. We avoid that a different way: we use greedy decoding everywhere except one cell, so there is no run-to-run noise to report. The exception is CheckRLM [51], whose own recipe samples at a temperature its authors set per backbone. Our standard is to keep an author's configuration where it differs from ours, so that cell decodes as they specify and the replication below does not cover it.

Rerunning one published cell that decodes greedily, three times from scratch with an identical command, produced **byte-identical results**: 72 numeric cells across 8 result rows, zero differences, the same hash after removing the timestamp. Error bars over a deterministic decoder would be zero-width and would say less than this does. The uncertainty that remains is sampling uncertainty, which the Wilson intervals [52] and the detectability threshold already carry.

What this does *not* show is insensitivity to item order, batch composition, or the few-shot examples a defense holds out. Those are untested, and a method whose result depended on them would pass this check and still be fragile.

Nor does it show reproducibility across environments, and we have a measured example of the difference. The defenses in the retrieval cell cannot share one Python environment: ACD's released decoder imports symbols that newer `transformers` releases removed, and its authors pin a version the rest of the cell cannot use. The *clean* arm plants no payload, so its result is a control. Re-running an earlier ACD cell under a rebuilt stack moved that arm by up to 0.3 pp. The closed-book arm moved too, and that arm never sees the context at all, so nothing but the environment can explain it. We therefore re-ran the earlier adaptive payload under the same stack as the gradient payload rather than comparing across the two; the attacked arms agreed to within 0.3 pp and the corrupted row agreed exactly. Determinism within a stack and reproducibility across stacks are different properties, and only the first is established by the replicates above.

We do not measure what the defenses cost. Every number here is an accuracy, a copy rate or a refusal rate. We report no inference latency, no memory footprint and no throughput, so nothing in this paper says whether a defense is affordable to deploy. That matters for the contrastive-decoding family in particular, which runs two forward passes per token. A method could be useless on the evidence here and still be the right choice at a cost we never measured, or hold up well and be unaffordable.

Single dataset per cell. The retrieval cell runs on CoConflictQA [13] and open Natural Questions [44]. The dosing model, which replaces a fraction of context with planted passages, is a simplification of a real retrieval pipeline under attack. It holds the retriever fixed, so it measures what the generator does with poisoned context rather than how hard the context is to poison.

Model coverage. Most retrieval-cell results use Llama-3-8B-Instruct, with Mistral-7B-Instruct where a defense required it. A second-model check left the results unchanged within the

evaluated range, but it covers only RobustRAG. The results should be read as holding for models of this class and size.

Scorer. We reuse each defense's own evaluation rubric where one exists, and a shared recall-based rubric across the retrieval cell. Recall against a gold string rewards a model that produces the right tokens for the wrong reason. We report the rate of answers from memory and the refusal rate alongside it so that the failure modes stay visible.

One screening stage is not reconstructible. Figure 2 counts each stage of corpus construction from recorded provenance. The step from the 10188 screened survivors to the 1158 papers read in full is the exception. The project kept no record of how that subset was chosen, so we cannot show it was unbiased. A reader should treat the corpus as a large and systematically discovered sample of the two literatures, not as their exhaustive intersection. The counts that follow from it are floors.

Most of the corpus is unrefereed, and the finding does not depend on that. 314 of 929 papers carry a named publication venue; the rest are preprints, and the share falls with recency, from 85% of our 2023 papers to 21% of our 2026 ones. A corpus restricted to refereed work would therefore describe an earlier field than the one we set out to measure, so the restriction is a check and not the corpus. Resolving both sides of every pair back to a paper succeeds for 292 of 349 pairs (84%); of those, 63 have a named venue on both sides, and 0 of them is a security defense evaluated against a safety failure mechanism. The empty direction stays empty among refereed work alone.

Census recall. 220 papers contributed no attack and no defense, and 140 of those match none of the survey, benchmark, framework or position descriptions by title. We re-read 143 papers — that bucket as it stood before the pass — and asked one question of each: does this paper name a technique we failed to register. 6 candidates came back. Two were genuine omissions and are now in the census. One was a defense already registered under a different name. One renamed a mechanism the registry carries three times. One was a payload form rather than a technique. The last was an attack a defense paper built for its own evaluation; Section III registers it under the rule that bars a paper's own attack from pairing with its own defense. Two omissions in 143 papers is a rate, not a guarantee of completeness.

To be done. Four items are not complete. We report none of them as complete.

- **Finish the campaign.** Some cells have not run. The counts above are a snapshot. The artifact lists every cell that remains.
- **Code the validation study.** The draw is complete. No coder has started. We give no extraction-error rate until the study returns.
- **Run the composition.** The experiment is queued. It puts one ingestion-stage method in front of one reasoning-stage method. The design rule stays a prediction until this run gives a result.
- **Read two published benefits from their papers.** ACD and DyPRAG give no figure in their text. Their controls stay directional.

IX. Related work

A. Systematizations of a single literature

Others have systematized the security side repeatedly and well. Work on the agent attack surface enumerates attack classes against tool-using systems [1], [16], including the skill and tool channels specifically [17]. Other work codes injection papers into structured comparison tables, by method, by surface, by victim goal and by capability [53], [54]. Others take a lifecycle or stack view of where an LLM system is exposed [55], [56]. Others trace long-term memory as its own attack surface [57]. Others survey agent trustworthiness broadly [58], [59]. All are careful. All of them stay inside the security literature.

The safety side has its own systematizations, and they do not overlap. One survey covers knowledge conflict between retrieved text and parametric memory on its own terms [18]. It treats the problem as one of accuracy, not of adversaries. That survey and the security ones above describe the same context window. They share almost no references, which Section III-D measures.

Three 2026 works sit closest to ours. A taxonomy-to-analysis study of the prompt-injection threat landscape [60] codes injection papers and builds a benchmark for agent behavior under context-dependent interaction. PIArena [61] is a modular platform for running injection attacks against injection defenses, and reports that defenses once judged effective lose robustness on more diverse attacks. A concurrent evaluation of injection defenses [62] finds that every defense which relies on the model to protect itself breaks under an adaptive attacker. That is the same direction as our Section VI-D result, reached on a disjoint set of methods. All three stay on the security side. That is the gap this paper measures, not a fault of theirs.

Against these, our contribution is not a better enumeration of either side. We apply one search protocol, one screening pass and one taxonomy to both. That makes the distance between them measurable, and it gives us the experiments in Section IV. A systematization drawn from one literature cannot report that 11.0% of security-regime papers cite a safety-regime one, or that 98.6% of tested pairs stay inside one regime. The other literature is not in its corpus.

B. Attack–defense evaluation platforms

The closest precedent is not a survey but a platform. DEEPSEC [63] ran 16 adversarial-example attacks against 13 defenses in one harness, which is structurally what we do here. It was also taken apart in public [50]. The criticisms are worth naming, because they define what such a platform has to get right. DEEPSEC evaluated defenses only against attacks crafted on undefended models. It reported robustness as an average over attacks, not against the strongest. It reimplemented several attacks and one defense incorrectly. And it reported no variance at all.

We took those as a specification. We do not reimplement: every method runs from its authors' released code and weights. The control-fires-first rule automates the check Carlini had to run by hand. It fired on FaithfulRAG, which we therefore report as not reportable rather than as defeated. We report per cell and never average across regimes. We also say where a cross-payload comparison is confounded. We use greedy decoding everywhere except the one cell whose authors specify sampling, so we report no error bars over a stochastic decoder. We verified that choice: a greedy cell rerun three times gives byte-identical output (Section VIII). The one criticism we inherit is obliviousness, and Sections IV and VI-D state its direction and test the single claim that needs it.

The reproducibility study of Olszewski et al. [64] is the methodological precedent for the other half. They sample papers from a venue, try to reproduce them, and report what failed and why. Our roadblock accounting in Section VII is in that genre. Several published defenses do not run from their released artifacts, which agrees with what they found.

The difference from both precedents is that we execute. Existing systematizations in this space report what papers claim their methods achieve. Section III-F shows why we do not. The side of the community that wrote the paper almost determines the reported outcome of a (defense, attack) pair. A systematization that aggregates those reports inherits that bias. Running the artifacts is the only way we found to avoid it.

X. Conclusion and future work

Two research communities study one failure. An LLM agent reads content it should not trust, and it cannot tell which content that is. The security community calls the content an attack and builds defenses. The safety community calls it a mechanism and builds fixes.

We measured the distance between them over 929 papers read in full, of which 709 name an attack or a defense. 11.0% of security-regime papers cite a safety-regime one. Only 14 of 541 defenses answer to both regimes. And 98.6% of tested (defense, attack) pairs stay inside a single regime. One direction of the cross-testing matrix holds 1 pair in 349: a security defense has been evaluated against a safety failure mechanism exactly once.

We then ran that experiment. 110 pairs, all on the authors' released artifacts. 33% of them cross the boundary the literature does not.

One poisoned passage in ten gives a defense its best chance, because nine honest passages survive, and it is the attacks' own default. 2 of 38 (defense, attack) pairs show a resolvable benefit there.

The finding is a mechanism, not a verdict. Context-reliance defenses do not fail because someone built them badly. They fail because of what they arbitrate. Each one decides between trusting the context and trusting memory, and that dial moves correctness around without creating any. Suppress memory and the attacker receives what memory held. Suppress the attacker and memory receives it back.

A payload that plants a third answer separates copying from remembering, so we can add the two movements together. In the three defenses that suppress memory they cancel to within 8 points. Neither direction moves anything toward the real document, because the dial has no setting for it. The attack is inside the context.

Two results bound that claim rather than soften it.

First, protection depends on the *form* of the planted passage, not only on whether an attacker is present. An evaluation built around optimized payloads therefore overstates how well a system holds. The fluent payload needs no gradient access and damages an undefended model roughly twice as hard.

Second, one defense did resist. It loses 58% of that protection to an attacker who reads its rule and optimizes against the quantity it measures.

Future work.

Extend the adaptive case to the rest of the family. We tested the one defense that resisted, with two optimizers. A best-of-five search over sentence forms took 29% of its protection. A gradient search over token ids took 58%. Together they settle that the defense degrades rather than collapses. Every other context-reliance method in the campaign has had only the non-adaptive treatment, which Section IV states is no evidence of robustness at all. The same two optimizers transfer directly: each of these methods exposes a scalar the attacker can address.

Close the supply-chain and cross-modal gaps. These channels have fewer than one named defense per named mechanism and are the routes by which agents accept third-party capability. They are the clearest place to build rather than to measure.

Separate the intervention-point finding from its confound. Whether a defense generalizes appears to follow where in the pipeline it intervenes. A confound sits beside that finding. The ingestion defenses that generalize are also the ones whose harnesses model a text-classification victim, and the execution defenses we could not evaluate are the agent-shaped ones. Running all three intervention points inside one agent harness would resolve it.

Make the empty cell the default, not the exception. The deepest problem this paper documents is not any single defense. It is that who runs an experiment predicts its outcome, and that nothing in current practice corrects for this. A shared harness would fix this. It should pair defenses with attacks that a taxonomy selects, not attacks that either author selects. Transfer then becomes a measured property rather than an assumed one. We release ours.

References

[1] K. Chu, "A systematic survey of security threats and defenses in llm-based ai agents: A layered attack surface framework," 2026.

[2] X. Ma, Y. Gao, Y. Wang, R. Wang, X. Wang, Y. Sun, Y. Ding, H. Xu, Y. Chen, Y. Zhao, H. Huang, Y. Li, Y. Wu, J. Zhang, X. Zheng, Y. Bai, Z. Wu, X. Qiu, J. Zhang, Y. Li, X. Han, H. Li, J. Sun, C. Wang, J. Gu, B. Wu, S.-M. Chen, T. Zhang, Y. Liu, M. Gong, T. Liu, S. Pan, C. Xie, T. Pang, Y. Dong, R. Jia, Y. Zhang, S. jie Ma, X. Zhang, N. Gong, C. Xiao, S. M. Erfani, T. Baldwin, B. Li, M. Sugiyama, D. Tao, J. Bailey,

and Y.-G. Jiang, "Safety at scale: A comprehensive survey of large model and agent safety," in *Found. Trends Priv. Secur.*, 2025.
[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *AISec@CCS*, 2023.
[4] W. Zou, R. Geng, B. Wang, and J. Jia, "Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models," in *USENIX Security*, 2024.
[5] Z. Zhong, Z. Huang, A. Wettig, and D. Chen, "Poisoning retrieval corpora by injecting adversarial passages," in *EMNLP*, 2023.
[6] P. Dash, T. Ge, A. Jain, T. A. Shah, and Z. Shang, "From untrusted input to trusted memory: A systematic study of memory poisoning attacks in llm agents," 2026.
[7] C. Xiang, T. Wu, Z. Zhong, D. Wagner, D. Chen, and P. Mittal, "Certifiably robust rag against retrieval corruption," 2024.
[8] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Gong, "Datasentinel: A game-theoretic detection of prompt injection attacks," in *IEEE S&P*, 2025.
[9] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji, "Ipiguard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents," in *EMNLP*, 2025.
[10] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," in *TACL*, 2023.
[11] S. Xia, Y. Wang, Z. Huang, and P. Liu, "Diagnosing and mitigating context rot in long-horizon search," 2026.
[12] J. Xie, K. Zhang, J. Chen, R. Lou, and Y. Su, "Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts," in *ICLR*, 2023.
[13] P. Huang, Z. Liu, Y. Yan, H. Zhao, X. Yi, H. Chen, Z. Liu, M. Sun, T. Xiao, G. Yu, and C. Xiong, "Parammute: Suppressing knowledge-critical ffns for faithful retrieval-augmented generation," in *NeurIPS*, 2025.
[14] B. Bi, S. Liu, Y. Wang, Y. Xu, J. Fang, L. Mei, and X. Cheng, "Parameters vs. context: Fine-grained control of knowledge reliance in language models," 2025.
[15] X. Yuan, Z. Yang, Y. Wang, S. Liu, J. Zhao, and K. Liu, "Discerning and resolving knowledge conflicts through adaptive decoding with contextual information-entropy constraint," in *ACL*, 2024.
[16] A. Dehghantanha and S. Homayoun, "Sok: The attack surface of agentic ai - tools, and autonomy," 2026.
[17] Y. Jiang, D. Li, H. Deng, B. Ma, X. Wang, Q. Wang, and G. Yu, "Sok: Agentic skills - beyond tool use in llm agents," 2026.
[18] R. Xu, Z. Qi, Z. Guo, C. Wang, H. Wang, H. Wang, and W. Xu, "Knowledge conflicts for llms: A survey," in *EMNLP*, 2024.
[19] D. Lee, M. Tiwari, and B. Miranda, "Prompt infection: Llm-to-llm prompt injection within multi-agent systems," in *ESORICS*, 2025.
[20] Z. Lin, H. Sun, and N. Shroff, "AI safety vs. AI security: Demystifying the distinction and boundaries," 2025.
[21] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Gong, and L. Sun, "Prompt injection attack to tool selection in llm agents," in *NDSS*, 2025.
[22] D. Schmotz, S. Abdelnabi, and M. Andriushchenko, "Agent skills enable a new class of realistic and trivially simple prompt injections," 2025.
[23] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. H. Chi, N. Scharli, and D. Zhou, "Large language models can be easily distracted by irrelevant context," in *ICML*, 2023.
[24] E. Debenedetti, J. Zhang, M. Balunovi'c, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate attacks and defenses for llm agents," in *NeurIPS*, 2024.
[25] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and D. Moher, "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, vol. 372, p. n71, 2021.
[26] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, "Prompt injection attack against llm-integrated applications," 2023.
[27] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases," in *NeurIPS*, 2024.
[28] Y. Chen, J. Arkin, Y. Zhang, N. Roy, and C. Fan, "Scalable multi-robot collaboration with large language models: Centralized or decentralized systems?" in *ICRA*, 2023.
[29] T. Zhu, Q. Liu, F. Wang, Z. Tu, and M. Chen, "Unraveling cross-modality knowledge conflicts in large vision-language models," 2024.
[30] J. Yan, V. Yadav, S. LI, L. Chen, Z. Tang, H. Wang, V. Srinivasan, X. Ren, and H. Jin, "Backdooring instruction-tuned large language models with virtual prompt injection," in *NAACL*, 2023.
[31] J. Yu, Y. Wu, D. Shu, M. Jin, and X. Xing, "Assessing prompt injection risks in 200+ custom gpts," 2023.
[32] T. Debi and W. Zhu, "Whispers of wealth: Red-teaming google's agent payments protocol via prompt injection," 2026.
[33] E. Feiglin, N. Hutnik, and R. Lapid, "Benchoverflow: Measuring overflow in large language models via plain-text prompts," in *Trans. Mach. Learn. Res.*, 2026.
[34] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," in *ICLR*, 2023.
[35] Z. Hong, L. Wang, L. Li, H. Ma, J. Fang, F. Shen, D. Zhang, and X. Wang, "Trace: Trajectory risk-aware compression for long-horizon agent safety," 2026.
[36] J. Xue, M. Zheng, Y. Hu, F. Liu, X. Chen, and Q. Lou, "Badrag: Identifying vulnerabilities in retrieval augmented generation of large language models," 2024.
[37] Y. Kim, H. J. Kim, C. Park, C. Park, H. Cho, J. Kim, K. M. Yoo, S. goo Lee, and T. Kim, "Adaptive contrastive decoding in retrieval-augmented generation for handling noisy contexts," in *EMNLP*, 2024.
[38] Y. Huang, S. Chen, H. Cai, and B. Dhingra, "To trust or not to trust? enhancing large language models' situated faithfulness to external contexts," in *ICLR*, 2024.
[39] A. Grattafiori *et al.*, "The llama 3 herd of models," 2024.
[40] A. Q. Jiang, A. Sablayrolles, A. Mensch *et al.*, "Mistral 7b," 2023.
[41] Y. Zhao, A. Devoto, G. Hong, X. Du, A. P. Gema, H. Wang, K.-F. Wong, and P. Minervini, "Steering knowledge selection behaviours in llms via sae-based representation engineering," in *NAACL*, 2024.
[42] R. Li, P. Huang, Z. Liu, Y. Yan, H. Xie, Y. Gu, G. Yu, and M. Sun, "Shift: Gate-modulated activation steering for knowledge conflict mitigation in retrieval-augmented generation," 2026.
[43] Qwen Team, "Qwen2.5 technical report," 2024.
[44] T. Kwiatkowski, J. Palomaki, O. Redfield *et al.*, "Natural questions: A benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019.
[45] Y. Ming, S. Purushwalkam, S. Pandit, Z. Ke, X.-P. Nguyen, C. Xiong, and S. Joty, "Faitheval: Can your language model stay faithful to context, even if "the moon is made of marshmallows"," in *International Conference on Learning Representations (ICLR)*, 2025.
[46] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "Squad: 100,000+ questions for machine comprehension of text," in *Empirical Methods in Natural Language Processing (EMNLP)*, 2016.
[47] Q. Zhang, Z. Xiang, Y. Xiao, L. Wang, J. Li, X. Wang, and J. Su, "Faithfulrag: Fact-level conflict modeling for context-faithful retrieval-augmented generation," in *ACL*, 2025.
[48] J. Su, J. X. Morris, P. Nakov, and C. Cardie, "Corpus poisoning via approximate greedy gradient descent," in *ACL*, 2024.
[49] R. A. Fisher, "On the interpretation of χ^2 from contingency tables, and the calculation of p," *Journal of the Royal Statistical Society*, vol. 85, no. 1, pp. 87–94, 1922.
[50] N. Carlini, "A critique of the DeepSec platform for security analysis of deep learning models," 2019.
[51] D. Xu, R. Wang, Q. Zhao, Y. Yan, Z. Wang, D. Zha, S. Yu, Z. Liu, S. Wang, X. Han, and M. Sun, "Checkrlm: Effective knowledge–thought coherence checking in retrieval-augmented reasoning," in *Volume 1*, 2026.
[52] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
[53] T. Yusifov, A. Aghayev, and L. Hasanzada, "Taxonomy of prompt injection attacks and analysis of defense mechanisms in large language model-based chatbots," in *InterConf*, 2026.
[54] J. D. Duarte, G. D. Cândido, J. R. A. D. B. Filho, J. S. Neto, E. J. da Costa, J. P. J. da Costa, and L. P. de Melo, "A systematic review of prompt injection attacks on large language models: Trends, taxonomy, evaluation, defenses, and opportunities," in *IEEE Access*, 2026.
[55] S. Natanzi and B. Tang, "A lifecycle and application-stack survey of large language model vulnerabilities: Attacks, risks, defenses, and open problems," 2026.

[56] T. Nemure, "Llm integration and the zero trust perimeter: A systematic review of trust enforcement failures and defensive architectures," in *World Journal of Advanced Research and Reviews*, 2026.
[57] Z. Lin, X. Hao, R. Fu, S. Cui, K. Chen, C. Li, Z. Li, and F. Xiong, "A survey on long-term memory security in llm agents: Attacks, defenses, and governance across the memory lifecycle," 2026.
[58] J. Qi, M. Li, J. Liu, Y. Shu, D. Yu, S. Ma, W. Cui, Y. Zhao, Y. Chen, R. Jiang, I. King, and Z. Xu, "Towards trustworthy agentic ai: a comprehensive survey of safety, robustness, privacy, and system security," 2026.
[59] K. Mohan and G. N. Srinivasa, "Agent security meets regulatory reality – a practitioner systematization of autonomous-agent threats and controls in regulated financial systems," 2026.
[60] P. Wang, X. Li, C. Xiang, J. Zhang, Y. Li, L. Zhang, X. Wang, and Y. Tian, "The landscape of prompt injection threats in llm agents: From taxonomy to analysis," 2026.
[61] R. Geng, C. Yin, Y. Wang, Y. Chen, and J. Jia, "Piarena: A platform for prompt injection evaluation," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026.
[62] P. Deep, S. Emmons, A. Fox, K. Bacon, K. McAllister, P. Ortiz, and K. Flautner, "Evaluation of prompt injection defenses in large language models," 2026.
[63] X. Ling, S. Ji, J. Zou, J. Wang, C. Wu, B. Li, and T. Wang, "DEEPSEC: A uniform platform for security analysis of deep learning model," in *IEEE Symposium on Security and Privacy*, 2019.
[64] D. Olszewski, A. Lu, C. Stillman, K. Warren, C. Kitroser, A. Pascual, D. Ukirde, K. Butler, and P. Traynor, ""get in researchers; we're measuring reproducibility": A reproducibility study of machine learning papers in tier 1 security conferences," in *ACM Conference on Computer and Communications Security (CCS)*, 2023, pp. 3433–3459.

Appendix